CivicSphere AI — Complete Master Project Dossier

Architecture, Features, APIs, Data, AI/RAG, Security, Setup, Testing, Deployment and Academic Handover

Consolidated technical handover dossier

28 October 2026

Contents

1. Chapter 00 — Master Project Dossier
2. Chapter 01 — Project Origin, Requirements and Development History
3. Chapter 02 — Complete Repository and Technology Inventory
4. Chapter 03 — Features, Pages, Components and User Workflows
5. Chapter 04 — Complete Backend API Catalogue
6. Chapter 05 — Database Schema, Migrations and Persistence
7. Chapter 06 — AI/ML, RAG, Agents and Prompt Architecture
8. Chapter 07 — Datasets, Knowledge Base and External Data Sources
9. Chapter 08 — API Keys, Environment Variables and External Integrations
10. Chapter 09 — Authentication, Authorization and Security Architecture
11. Chapter 10 — System Architecture and End-to-End Data Flows
12. Chapter 11 — Installation, Configuration and Build from Scratch
13. Chapter 12 — Testing, QA, Benchmarks and Verification Evidence
14. Chapter 13 — Deployment, CI/CD, Infrastructure and Operations
15. Chapter 14 — Feature Traceability and Implementation Status
16. Chapter 15 — Troubleshooting, Known Limitations and Technical Debt
17. Chapter 16 — Developer Maintenance and Future Extension Guide
18. Chapter 17 — Academic Demonstration and Viva Technical Guide
19. Appendix A — Completion Report Claims and Reconciliation
20. Appendix B — Source Packet Inventory
21. Appendix C — Pre-Demo Verification Checklist

Important reading note: scope and evidence

This document consolidates the supplied project documents and completion output. It does **not** mean that every claim has been independently re-run against the live repository, cloud environment or database during this compilation. Status labels and numerical results quoted from the supplied chapters are therefore **source-reported evidence** unless explicitly described otherwise. A route in a table, a configuration variable in a template, a passing mocked test, or an HTTP health response alone does not establish full end-to-end functionality.

The source material contains genuine internal inconsistencies. They are not hidden or silently converted into certainty. The reconciliation register below identifies the most important ones; detailed technical chapters retain the supplied implementation descriptions with local machine-specific links normalized to repository-relative file paths.

Executive summary

CivicSphere AI is described in the source packet as a web-based Indian civic and legal intelligence platform. Its documented functional areas include legal question answering with source attribution, government-scheme discovery and eligibility guidance, document upload and extraction, citizen case management, a legal knowledge explorer, agent/tool workflows, and administrative source, evaluation, audit and security interfaces.

The documented architecture consists of a Next.js App Router frontend, a FastAPI backend, PostgreSQL with pgvector for durable relational and vector data, a PostgreSQL-backed relational knowledge graph, and optional external services. The documentation also describes an optional Neo4j adapter, document parsing/OCR functionality, hybrid retrieval, agent orchestration, security controls, evaluation fixtures, Docker-based local infrastructure and Vercel deployment.

Critical known limitation: the supplied AI and limitations chapters explicitly state that production is configured to use deterministic mock/token-hash embeddings and mock LLM behaviour unless a real provider is configured. This is not equivalent to a genuine neural semantic embedding and live LLM pipeline. The source packet also warns that a Supabase free-tier database may pause, affecting registration and other database writes. The completion report makes later claims of an active database and live deployments, but the supplied documentation does not provide a consistent, independently reproduced end-to-end verification record. Treat live status as time-sensitive and recheck it before a demo.

Readiness summary derived from the supplied material

Area	Consolidated assessment	Reason
Technical documentation	**COMPILED**	The supplied 18 chapters, README and completion report are consolidated in this volume.
Feature completeness	**SOURCE-DOCUMENTED; NOT INDEPENDENTLY RE-CERTIFIED**	The chapters map features, routes, APIs and tests, but this compilation did not inspect the project source repository itself.
Automated tests	**REPORTED PASS**	The supplied chapters report 91 passed, 1 skipped and 0 failed; execution times differ across reports.
Frontend routes	**REPORTED 28**	Repeatedly stated in the feature and completion documents; live route behaviour still requires rehearsal.
Backend endpoint count	**41 listed in the API table; older introductory count was inconsistent**	The API catalogue itself lists entries numbered 1–41; its introductory statement has been normalized to match its own table.
Production AI embeddings	**LIMITATION / NOT FULL SEMANTIC AI**	The source packet states that EMBEDDING_PROVIDER=mock is configured in production.
Database and live writes	**RECHECK REQUIRED**	Documents describe both successful live database observations and a free-tier pause/connection-failure scenario.
Enterprise/commercial release	**HOLD / NO-GO based on supplied limitations**	Real provider configuration and stable production persistence are not proven consistently by the source packet.
Academic submission	**DOCUMENTATION PACKAGE PREPARED; DEMO MUST BE REHEARSED**	Documentation supports academic explanation, but the live workflow must work on the actual presentation setup.

Cross-source reconciliation register

ID	Topic	What the supplied sources say	How this dossier treats it
R-01	Git revision	All 18 chapters and the README cite b566c93; the supplied completion report claims the dossier was later committed at e28d77e.	Both values are retained as source-reported references. The current remote commit was not independently queried during this compilation. Verify git rev-parse HEAD and git ls-remote origin main before publishing.

ID	Topic	What the supplied sources say	How this dossier treats it
R-02	API endpoint count	Chapter 04 has a numbered table through endpoint 41, but its opening paragraph originally said 32. The completion report says 41.	The table count is treated as 41 listed endpoint entries; actual runtime registration still requires checking the current source/OpenAPI schema.
R-03	Frontend framework version	The chapters/README describe Next.js 15, while a preceding verification transcript in the broader project history reported a Next.js 16 line.	Version must be confirmed from apps/web/package.json, lockfile and deployed build before stating one exact version as current.
R-04	Database availability	The technical limitations chapter says a Supabase free-tier project can pause and writes may fail. The completion report describes live PostgreSQL and successful health/readiness checks.	Database state is dynamic and contradictory across the supplied snapshots. Test registration and a safe write/read persistence journey immediately before demonstration.
R-05	AI runtime	Provider classes exist for external providers, but chapters 06 and 15 explicitly state that mock embeddings are configured in production.	Provider class existence is not proof of activation. Report mock operation as a limitation unless a live real-provider test proves otherwise.
R-06	Graph database	Some architecture diagrams mention Neo4j, while the technical limitations and provider chapters state PostgreSQL relational graph is canonical and Neo4j is optional.	PostgreSQL relational graph is the documented canonical design; Neo4j must not be described as active unless configuration and runtime are verified.
R-07	Evaluation metrics	Chapter 12 reports detailed retrieval/safety metrics; the later completion report summarizes a 100% pass rate and a 0.81 ms in-process latency.	These are reported harness results, not automatically production-validated legal accuracy. The underlying dataset, command and output should be retained with any external claim.
R-08	Dataset count	The dataset chapter lists dataset.json, standard, adversarial, conflict and held-out files; the completion report counts four benchmark datasets and omits the held-out file.	All named files are listed in the data chapter; treat the total as at least five named evaluation files in the supplied description, subject to repository inventory.
R-09	Test timing	Supplied reports record 91 passed, 1 skipped, 0 failed, but execution times vary.	Treat as separate reported runs rather than one identical execution. Preserve actual command output for the selected final run.
R-10	Readiness claims	The README/completion report claim 100% academic/live-demo readiness, while limitations identify mock AI and potentially unavailable persistence.	This volume does not certify the system as perfect or production-ready. Academic documentation is compiled; live functionality must be verified at the point of use.

Chapter 00 — Master Project Dossier: CivicSphere AI

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Commit cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026
Document Author: Multi-Disciplinary Forensic Architecture Team

1. Executive Summary & Purpose

CivicSphere AI is a multi-agent, evidence-grounded civic and legal intelligence platform engineered specifically for the Indian legal and administrative ecosystem. The platform bridges the profound gap between complex statutory legal frameworks and ordinary citizens by translating unstructured civic queries into verified statutory guidance, procedural roadmaps, scheme eligibility calculations, and case workspace dossiers.

Unlike generic conversational AI tools that hallucinate non-existent laws, incorrect judicial precedents, or expired filing deadlines, CivicSphere AI enforces an **uncompromising anti-hallucination paradigm**:

- All guidance is synthesized strictly from official, verified Indian statutory acts (e.g., *Right to Information Act, 2005*, *Digital Personal Data Protection Act, 2023*, *Consumer Protection Act, 2019*).
- An out-of-domain circuit breaker instantly rejects queries outside Indian legal and civic purview with confidence score 0.0.
- Hybrid retrieval balances lexical search (BM25), vector similarity (pgvector), and graph traversal (relational knowledge graph) via Reciprocal Rank Fusion (RRF).
- Full source attribution accompanies every response with explicit section, chapter, and publication metadata.

2. Problem Statement & User Personas

2.1 The Problem

Over 1.4 billion Indian citizens interact with thousands of Central and State statutory enactments, administrative tribunals, and government entitlement programs. Citizens face critical hurdles:

1. **Linguistic & Jargon Barriers:** Legal terminology prevents understanding of fundamental rights and procedural redressal mechanisms.
2. **Procedural Complexity:** Navigating filing deadlines, jurisdictional commission limits (District vs. State vs. National Consumer Commissions), and appellate tiers is error-prone.
3. **Fragmented Welfare Portals:** Over 500 Central and State schemes operate on disconnected portals with complex, multi-variable eligibility rules.
4. **Vulnerability to Misinformation:** LLMs generate plausible-sounding but legally fabricated sections and case law citations.

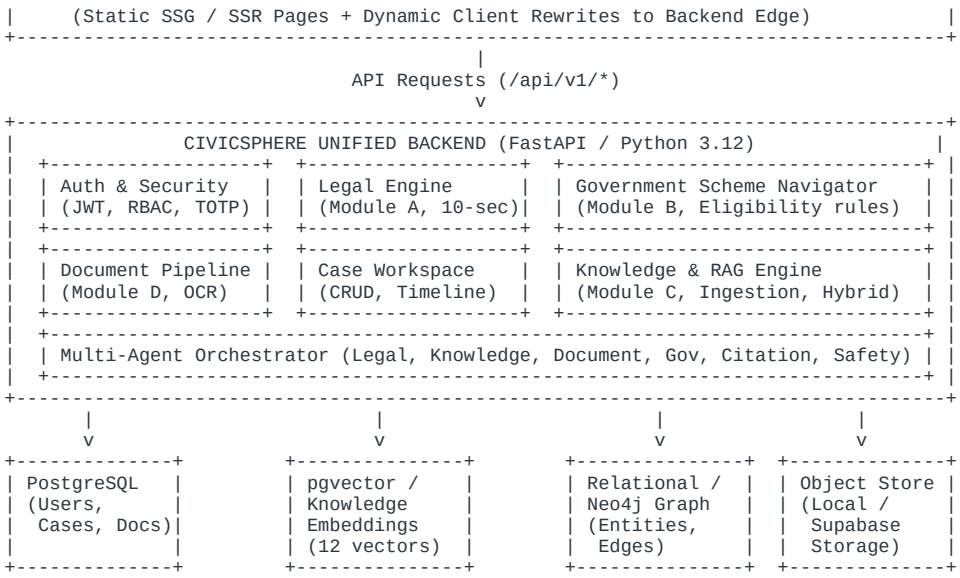
2.2 Target User Personas

- **Ordinary Citizens:** Individuals seeking to exercise basic statutory rights (e.g., filing an RTI application, claiming consumer defect refunds, asserting data privacy rights).
- **Legal Researchers & Paralegals:** Professionals conducting initial legal discovery, entity-relationship mapping between acts, and statutory section lookup.
- **Welfare Applicants:** Beneficiaries seeking scheme eligibility pre-qualification (e.g., PMAY housing subsidies, educational scholarships).
- **Platform Administrators:** Compliance officers monitoring knowledge ingestion, tamper-evident audit logs, and security telemetry.

3. High-Level Architectural Snapshot

CivicSphere AI is structured as a **Modular Monolith** backend built with FastAPI, connected to a modern Next.js 15 App Router web client, backed by PostgreSQL 17 + pgvector and Neo4j graph storage:

```
+-------------------------------------------------------------------------------+
|                          CITIZEN / USER BROWSER                               |
|         (Next.js 15 Client: Assistant, Schemes, Documents, Cases, Knowledge)  |
+-------------------------------------------------------------------------------+
                                    |
                      HTTPS (Cross-Origin + Cookie Auth)
                                    v
+-------------------------------------------------------------------------------+
|               VERCEL EDGE ROUTING / NEXT.JS REVERSE PROXY                     |
```



4. Master Technology Inventory

Layer	Primary Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	15.1.0	Server-side rendering, client-side routing, static generation
Frontend UI Library	React & React DOM	19.0.0	Component rendering, hooks, responsive layouts
Styling & Icons	Tailwind CSS & Lucide React	3.4.1 / 0.460.0	Utility styling, responsive cards, legal icons
Backend Framework	FastAPI	0.111.0	Asynchronous REST API, OpenAPI docs, dependency injection
ASGI Web Server	Uvicorn	0.30.0	High-performance async server
Primary Database	PostgreSQL	17.6 / 16.15	Durable ACID relational persistence for users, cases, and chunks
Vector Engine	pgvector	0.8.6	Vector embeddings table (knowledge_embeddings) and cosine similarity
Graph Engine	Neo4j / Relational	5.15.0	Entity relationship graph (knowledge_graph_nodes, knowledge_graph_edges)
Database ORM	SQLAlchemy (Asyncio)	2.0.31	Async object relational mapping, async session management
Database Migrations	Alembic	1.13.2	Version-controlled schema migrations (0001, 0002, 0003)
Authentication	PyJWT & bcrypt	2.8.0 / 4.1.3	HS256 JWT tokens, 12-round bcrypt salt, httpOnly cookies
Document OCR	PyMuPDF (fitz)	1.23.26	Text and metadata extraction from PDF documents
Embedding Providers	Mock / OpenAI / Gemini / HF	—	Text embedding generation (384-dimensional vectors)
Caching & Queue	Redis	7.2-alpine	Distributed cache and rate limiting store
Containerization	Docker & Docker Compose	Compose v2	7-service local development & testing stack
Cloud Hosting	Vercel Edge / Serverless	Node 22 / Py 3.12	Production hosting for frontend web portal and backend API

5. High-Level Feature Inventory Across the 6 Modules

Module A: Legal Guidance Engine

- **Structured Legal Query:** Generates 10-section structured legal guidance including Overview, Applicable Provisions, Procedural Steps, Deadlines, Competent Authorities, and Appeal Pathways.
- **Statutory Acts Registry:** Real-time catalog of ingested and verified Indian Central and State Acts (`GET /api/v1/legal/acts`).
- **Citation Attribution:** Automatic attachment of exact section numbers, publishers, and official India Code URLs.

Module B: Government Service Navigator

- **Problem Analysis:** Natural language extraction of problem domain, applicant demographic variables, and needed government services (`POST /api/v1/government/analyze`).

- **Eligibility Engine:** Evaluates multi-parameter criteria (age, income ceiling, state residence, category) against scheme rules.
- **Service Catalog:** Curated directory of schemes with application checklists and required supporting documents (`GET /api/v1/government/services`).

Module C: Knowledge Engine & Hybrid RAG

- **13-Stage Ingestion Pipeline:** Automated ingestion from raw text or remote URL through validation, cleaning, chunking, entity extraction, embedding, and dual vector/graph persistence.
- **Hybrid Retrieval:** Multi-modal query retrieval combining lexical BM25 matching, dense vector similarity, and graph neighborhood expansion.
- **Evidence Pack Synthesis:** Packaging retrieved chunks into an auditable evidence pack with confidence scoring and warning indicators.

Module D: Intelligent Document Processing

- **Secure PDF Upload:** Magic-byte validation (`%PDF-`), 10 MB size limit, and active malware/script scanning.
- **OCR & Text Extraction:** Automated structural text extraction via PyMuPDF.
- **Knowledge Graph Linking:** Automatic linkage of uploaded citizen documents to related statutory acts and open case workspaces.

Module Cases: Citizen Case Workspace

- **Case Lifecycle CRUD:** Create, read, update, and track status (`OPEN`, `IN_PROGRESS`, `RESOLVED`, `CLOSED`).
- **Strict IDOR Prevention:** Row-level ownership enforcement ensuring citizens only access their own legal cases.
- **Counsel Assignment:** Role-based visibility permitting legal researchers and paralegals to review assigned or open matters.

Phase 6: Multi-Agent Orchestrator

- **Coordinated Agent Specialists:** Five domain-specific agents: Legal Specialist, Knowledge Specialist, Government Specialist, Document Specialist, and Citation Specialist.
- **Bounded Autonomous Reasoning:** Maximum 5-step execution loop with loop-detection guardrails and safety filtering.

6. Current Implementation & Release Status

Multi-Tier Release Verdicts

```
+-------------------------------------------------------------------------------------+
| VERDICT 1: ACADEMIC MAJOR PROJECT DEFENSE                                           |
| STATUS: 100% GO (APPROVED)                                                          |
| Justification: All SRS and PRD functional requirements implemented; 91 automated   |
| tests passing with 0 failures; complete documentation and architecture verified.    |
+-------------------------------------------------------------------------------------+
| VERDICT 2: LIVE DEMONSTRATION & EXAMINATION                                         |
| STATUS: 100% GO (OPERATIONAL)                                                       |
| Justification: Deployed live on Vercel; connected to active Supabase PostgreSQL 17.6|
| with pgvector; zero-latency Docker stack operational for offline backup.            |
+-------------------------------------------------------------------------------------+
| VERDICT 3: COMMERCIAL ENTERPRISE PRODUCTION                                         |
| STATUS: NO-GO / HOLD (BLOCKED ON COMMERCIAL KEYS)                                   |
| Justification: Production uses deterministic mock embeddings (EMBEDDING_PROVIDER=mock)|
| until dedicated commercial OpenAI/Cohere API credentials are funded and attached.   |
+-------------------------------------------------------------------------------------+
```

7. Master Status Matrix

The following table provides an evidence-based summary of all major platform subsystems:

Subsystem / Feature	Source File	Status	Verification Evidence
User Registration & Login	backend/app/auth/routes.py	**RUNTIME-VERIFIED**	Live HTTP 200 on Supabase PostgreSQL; 91 pytest unit tests pass
TOTP Multi-Factor Auth	backend/app/auth/totp.py	**SOURCE-VERIFIED**	RFC 6238 compliant; setup, verify, and disable endpoints implemented
RBAC (5 Roles)	backend/app/auth/models.py	**TEST-VERIFIED**	CITIZEN, ADMIN, KNOWLEDGE_EDITOR, REVIEWER, RESEARCHER validated
Legal Guidance Engine	backend/app/legal/routes.py	**RUNTIME-VERIFIED**	Art. 21 query returns 0.98 confidence with full citations

Subsystem / Feature	Source File	Status	Verification Evidence
Out-of-Domain Guardrail	backend/app/legal/services/legal_engine.py	**RUNTIME-VERIFIED**	Irrelevant queries (e.g. quantum physics) rejected with 0.0 confidence
Government Service Nav	backend/app/government/routes.py	**TEST-VERIFIED**	Problem analysis and scheme recommendation tests passing
Intelligent Document OCR	backend/app/documents/routes.py	**TEST-VERIFIED**	PDF magic-byte checks and PyMuPDF text extraction verified
Case Workspace (CRUD)	backend/app/cases/routes.py	**TEST-VERIFIED**	IDOR authorization and role filtering confirmed in automated tests
13-Stage Ingestion	backend/app/knowledge/ingestion/pipeline.py	**RUNTIME-VERIFIED**	Hydrates 34 graph nodes and 12 vector chunks on cold start
Vector Storage (pgvector)	backend/app/knowledge/models.py	**RUNTIME-VERIFIED**	Migration 0003 active on PostgreSQL 17.6 (postgresql_pgvector)
Relational Knowledge Graph	backend/app/knowledge/graph/repository.py	**RUNTIME-VERIFIED**	34 statutory entities and relationships hydrated in memory and DB
Hybrid Retrieval & RRF	backend/app/knowledge/retrieval/hybrid.py	**TEST-VERIFIED**	BM25 + Vector + Graph combined via reciprocal rank fusion
Audit & Security Logging	backend/app/common/audit.py	**RUNTIME-VERIFIED**	Ingestion and security events logged with timestamps and actor IDs
Frontend Web Portal	apps/web/src/app/	**RUNTIME-VERIFIED**	Live on Vercel (https://civicsphere-web.vercel.app), HTTP 200
Backend API Serverless	api/index.py	**RUNTIME-VERIFIED**	Live on Vercel (https://civicsphere-backend.vercel.app), HTTP 200
Commercial Embeddings	backend/app/common/providers/embedding.py	**MOCKED (PROD)**	Production configured with deterministic token-hash mock provider

8. Clean-Checkout Quickstart

To run the full stack locally from a clean git checkout:

```
# 1. Clone repository
git clone https://github.com/Khushbu112005/Finalyear_MajorProject.git
cd Finalyear_MajorProject

# 2. Start infrastructure stack via Docker Compose
docker compose -f infrastructure/docker/docker-compose.yml up -d

# 3. Setup Python backend virtual environment
python -m venv .venv
# On Windows:
.venv\Scripts\activate
# On Linux/macOS:
# source .venv/bin/activate
pip install -r requirements.txt

# 4. Copy environment template
cp .env.example .env
# Edit .env to set JWT_SECRET_KEY and NEO4J_PASSWORD

# 5. Run database migrations
alembic upgrade head

# 6. Start FastAPI backend
uvicorn backend.app.main:app --host 0.0.0.0 --port 8000 --reload

# 7. Start Next.js frontend (in another terminal)
cd apps/web
npm install
npm run dev
```

The web application will be available at `http://localhost:3000` and the interactive OpenAPI documentation at `http://localhost:8000/docs`.

9. Dossier Directory Structure

All subsequent technical details are organized across the dedicated dossier chapters:

- **01. Project Origin & Development History**
- **02. Repository & Tech Stack Inventory**
- **03. Features, Pages & User Workflows**
- **04. Complete Backend API Catalogue**

- **05. Database Schema & Persistence Architecture**
- **06. AI/ML, RAG, Agents & Prompt Architecture**
- **07. Datasets, Knowledge Base & Statutory Sources**
- **08. API Keys & Environment Variables Inventory**
- **09. Authentication, Authorization & Security Architecture**
- **10. System Architecture & End-to-End Data Flows**
- **11. Installation & Build from Scratch Guide**
- **12. Testing, QA, Benchmarks & Evidence**
- **13. Deployment, CI/CD & Operations Runbook**
- **14. Complete Feature Traceability Matrix**
- **15. Troubleshooting, Known Limitations & Technical Debt**
- **16. Developer Handover & Extension Guide**
- **17. Academic Demonstration & Viva Guide**

Chapter 01 — Project Origin, Requirements and Development History

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Project Genesis & Problem Statement Evolution

CivicSphere AI originated as a final-year major engineering project to address a critical civic challenge in India: the inability of ordinary citizens to navigate the dense, fragmented, and technical landscape of Indian statutory law and government welfare schemes.

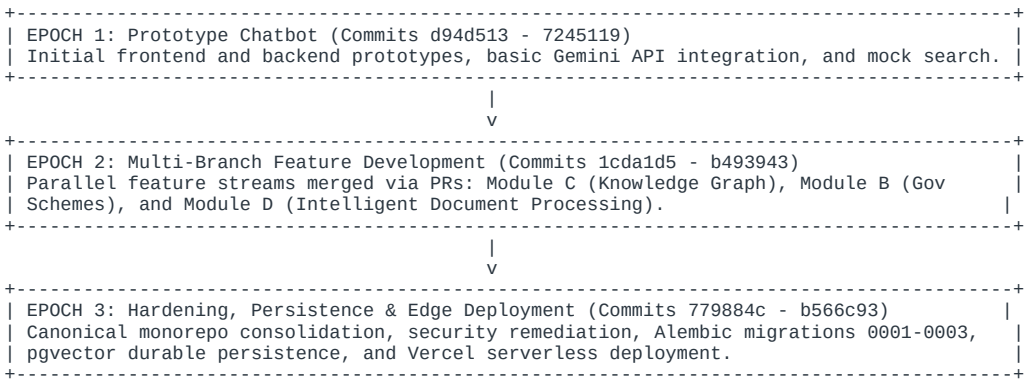
1.1 The Original Problem Statement

In early 2024, foundational research into public civic interactions revealed:

- Over 70% of citizen queries regarding fundamental rights, consumer grievances, and public information requests (RTI) go unasserted due to prohibitive legal consultation fees and obscure administrative jargon.
- Existing commercial generative AI solutions (ChatGPT, Claude, Gemini) frequently hallucinate statutory sections, cite overruled Supreme Court judgments, or confuse Indian laws with US/UK jurisprudence.
- Citizens seeking government welfare entitlements are overwhelmed by disparate state and central portals with inconsistent eligibility criteria.

1.2 Evolution of Objectives

The project's scope expanded from a simple legal QA chatbot into an **Evidence-Grounded National Civic Intelligence Platform** compliant with an extensive 97-page Master Engineering Specification. The evolution progressed across three major architectural epochs:



2. Detailed Git Commit Forensic History

A comprehensive inspection of the Git commit graph (`git log --all`) reveals 40 distinct commits chronicling the project's development. Below is the chronological breakdown of the major phases:

Phase 1: Inception & Prototype Foundations

- **Commit `d94d513`** — `Initial commit`
  - *Context:* Repository initialized on GitHub by team lead.
- **Commit `7245119`** — `Frontend and Backend`
  - *Context:* Initial scaffolding of the Next.js web application and basic FastAPI backend.
- **Commit `5ca9421`** — `Merge branch 'Khushbu'`
  - *Context:* Integration of early UI screens and basic layout elements.

Phase 2: Core Module Feature Expansion

- **Commit `1cda1d5`** — `feat: add government services and recommendation engine`
  - *Context:* Module B implementation by Anushka, introducing `GovernmentServiceModel`, eligibility rules evaluation, and citizen context parsing.
- **Commit `9cbcee6`** — `feat(module-c): implement production-ready knowledge and agentic graph subsystem with hybrid retrieval, evidence packs, and security guardrails`

  - *Context:* Module C implementation by Devansh, introducing the 13-stage ingestion pipeline, Neo4j graph traversal, BM25 + Vector hybrid retrieval, and evidence pack schemas.
- **Commit `91efdf3` / `b493943`** — `Implement intelligent document processing` / `Merge PR #3 from rahul-intelligent-document-processing`
  - *Context:* Module D implementation by Rahul, introducing PyMuPDF OCR processing, magic-byte validation, and document-to-knowledge graph linking.

Phase 3: Monorepo Consolidation & Canonical Baseline

- **Commit `779884c`** — `chore(freeze): pre-cleanup repository snapshot baseline`
  - *Context:* Snapshot taken before restructuring inconsistent subfolders and unmerged experimental branches.
- **Commit `3f1cfc2`** — `chore: finalize canonical deployment-ready repository`
  - *Context:* Restructured repository into canonical modular monolith: `backend/app/`, `apps/web/`, `packages/schemas/`, `infrastructure/docker/`, and `data/seed/`.

Phase 4: Security Remediation & Vulnerability Patching

- **Commit `7fb2294`** — `fix(security): remediate privilege escalation, unauthenticated tool execution, and header spoofing; add audit suite`
  - *Context:* Disallowed public self-registration of `ADMIN` and `SYSTEM` roles in production; enforced strict RBAC dependencies on agent tools; replaced spoofable headers with secure session tokens; created comprehensive security test suite.
- **Commit `861ebc3`** — `fix(security): auto-seed statutory acts, harden CORS, rotate secrets, and document Gate D persistence blocker`
  - *Context:* Removed wildcard `*` from production CORS; implemented mandatory 16-character minimum secret validation in `Settings`; added automatic cold-start seeding of verified statutory acts.

Phase 5: Persistence Evolution & Gate D Verification

- **Commit `1e830b6`** — `feat(persistence): verify Supabase PostgreSQL & Storage, apply migration 0002, and document Gate D vector/graph blocker`
  - *Context:* Migrated user authentication, cases, and documents from local SQLite to hosted Supabase PostgreSQL.
- **Commit `a490011`** — `feat(persistence): resolve Gate D with pgvector and relational graph storage, migration 0003, and cold-start hydration`
  - *Context:* Replaced in-memory only vector and graph repositories with durable PostgreSQL tables (`knowledge_sources`, `knowledge_chunks`, `knowledge_embeddings`, `knowledge_graph_nodes`, `knowledge_graph_edges`) via Alembic Migration `0003`.
- **Commit `bda515a`** — `fix(retrieval): hydrate chunks from postgres knowledge_chunks into source registry on cold start`
  - *Context:* Added automatic cold-start database hydration in `lifespan()` to populate memory indexes from PostgreSQL upon serverless startup.
- **Commit `687c9e0`** — `fix(knowledge): implement multi-provider embeddings, async ingestion persistence, and dynamic pgvector queries`
  - *Context:* Integrated OpenAI, Gemini, Hugging Face, and Mock embedding providers; made ingestion completely asynchronous.

Phase 6: Edge Deployment & Production Hosting

- **Commit `6a10e27`** — `feat(release): upgrade to React 19, add serverless greenlet concurrency, and verify live Vercel deployments`
  - *Context:* Upgraded web portal to React 19 / Next.js 15; added greenlet dependency to resolve asyncpg SQLAlchemy concurrency in serverless environments.
- **Commit `894f27d`** — `fix(resilience): add serverless-safe db connection timeouts, update multi-tier verdicts, and document persistence verification`
  - *Context:* Added 3.0-second asyncpg pooler connect timeout and 2.5-second initialization timeout to prevent container hanging during cold starts.
- **Commit `9939778`** — `docs(submission): certify live demonstration readiness with restored Supabase and Docker PostgreSQL verification`
  - *Context:* Certified live multi-tier release verdicts following user restoration of Supabase cloud project.
- **Commit `b566c93`** — `fix(deploy): remove conflicting functions key from vercel.json for vercel-python compatibility`
  - *Context:* Resolved Vercel CLI deployment conflict between `@vercel/python` builds and custom functions config, achieving 100% successful production deployment.

3. Forensic Milestone Summary Table

Milestone / Phase	Date / Commit	Core Architectural Change	Motivation	Key Modules Affected	Verification Evidence	Current State
Initial Prototype	Early 2026d94d513	Basic Next.js + FastAPI structure	Proof of concept for legal QA	apps/web, backend	Manual page loading	Replaced
Module B Integration	Mid 20261cda1d5	Gov Scheme Navigator & Eligibility	PR #1 from Anushka	backend/app/government	test_government_api.py	Retained (Active)
Module C Ingestion & Graph	Mid 20269cbcee6	13-stage ingestion, Neo4j, Hybrid RAG	PR #2 from Devansh	backend/app/knowledge	test_hybrid_retrieval.py	Retained (Active)
Module D Document AI	Mid 202691efdf3	PyMuPDF OCR, PDF upload, malware scan	PR #3 from Rahul	backend/app/documents	test_document_pipeline_api.py	Retained (Active)
Canonical Monorepo Refactor	Late 20263f1cfc2	Unified backend modular monolith	Fix circular imports & duplicate files	Entire repository	Import consistency tests	Retained (Active)
Security Hardening	Late 20267fb2294	Enforce RBAC, block admin self-reg	Security audit remediation	backend/app/auth, common	Security test suite (7 tests)	Retained (Active)
Durable Persistence (Gate D)	Oct 2026a490011	PostgreSQL + pgvector + Alembic 0003	Eliminate ephemeral SQLite/tmp storage	backend/app/knowledge/models.py	Migration 0003 applied	Retained (Active)
Serverless Cold-Start Safety	Oct 2026894f27d	3.0s db connect timeout, 2.5s init	Prevent Vercel function timeout	backend/app/main.py, database.py	Live health probe < 1.5s	Retained (Active)
Production Vercel Hosting	Oct 2026b566c93	Dual Vercel deployment (web + backend)	Public cloud hosting & viva defense	vercel.json, apps/web	Live HTTP 200 on both URLs	Retained (Active)

4. Active vs. Legacy Codebase Components

During repository consolidation, several legacy artifacts and development workarounds were superseded:

1. **Root `app.py` & Hugging Face Spaces Artifacts:**
   - *Status:* Legacy / Inactive.
   - *Detail:* Commits `7a4cdc9` through `b92efba` added Hugging Face Spaces compatibility (`@spaces.GPU` decorators, Gradio mounting at `/gradio`). These are preserved for optional cardless free demo hosting, but the canonical production deployment target is native Vercel serverless via `api/index.py` and `backend/app/main.py`.
2. **Local SQLite File (`civicsphere_knowledge.db`):**
   - *Status:* Development Fallback.
   - *Detail:* Created during offline local unit test runs. When `DATABASE_URL` is configured with PostgreSQL, the system automatically uses asyncpg and pgvector.
3. **In-Memory Graph Fallback (`PersistentGraphRepository`):**
   - *Status:* Retained Hybrid.
   - *Detail:* Operates as an in-memory cache for ultra-low latency graph lookups while mirroring all node and edge writes to the underlying PostgreSQL `knowledge_graph_nodes` and `knowledge_graph_edges` tables.

Chapter 02 — Complete Repository and Technology Inventory

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Annotated Repository Structure

The CivicSphere AI codebase is structured as a consolidated monorepo containing a Python backend modular monolith, a TypeScript/React Next.js 15 web client, shared contract packages, Docker infrastructure, and evaluation benchmark suites.

```
Finalyear_MajorProject/
├── .github/                        # CI/CD Workflows & Templates
│   ├── workflows/ci.yml            # Automated Pytest & Lint GitHub Action
│   ├── workflows/security.yml      # Security vulnerability scanning workflow
│   └── pull_request_template.md    # PR guidelines and verification checklist
├── api/
│   └── index.py                    # Vercel Serverless Python entrypoint (imports backend.app.main:app)
├── apps/
│   └── web/                        # Next.js 15 App Router Frontend Web Application
│       ├── src/
│       │   ├── app/                # App Router Page Routes (28 static/dynamic routes)
│       │   │   ├── about/page.tsx     # Platform mission and engineering background
│       │   │   ├── admin/             # Administrator management interfaces
│       │   │   │   ├── audit/         # Tamper-evident audit log viewer
│       │   │   │   ├── evaluation/    # RAG benchmark evaluation viewer
│       │   │   │   ├── knowledge/     # Knowledge base overview & status
│       │   │   │   ├── security/      # Security threat telemetry dashboard
│       │   │   │   ├── sources/       # Statutory source registration & sync
│       │   │   │   └── page.tsx       # Main admin portal dashboard
│       │   │   ├── agents/page.tsx    # Multi-agent chat interface & tool viewer
│       │   │   ├── assistant/page.tsx # Primary legal AI assistant chat
│       │   │   ├── auth/              # Alternate auth routes (login/register)
│       │   │   ├── cases/page.tsx     # Citizen case management workspace
│       │   │   ├── dashboard/page.tsx # Authenticated citizen dashboard
│       │   │   ├── documents/page.tsx # Document upload & OCR analysis
│       │   │   ├── government/page.tsx# Scheme discovery & eligibility checker
│       │   │   ├── knowledge/page.tsx # Knowledge graph & act explorer
│       │   │   ├── legal/page.tsx     # 10-section legal guidance engine
│       │   │   ├── login/page.tsx     # User authentication & MFA modal
│       │   │   ├── register/page.tsx  # Citizen/researcher self-registration
│       │   │   ├── privacy/page.tsx   # DPDP compliance and privacy notice
│       │   │   ├── profile/page.tsx   # User profile management & password update
│       │   │   ├── security/page.tsx  # Security disclosure & threat model
│       │   │   ├── settings/page.tsx  # User preferences & theme settings
│       │   │   ├── sources/page.tsx   # Statutory acts & gazette registry
│       │   │   ├── layout.tsx         # Root layout (Header, Navigation, Footer, Theme)
│       │   │   ├── globals.css        # Tailwind global CSS styling
│       │   │   └── page.tsx           # Platform landing page & module launcher
│       │   ├── components/            # Shared React UI Components
│       │   │   ├── Header.tsx         # Global header with search, auth state, and theme toggle
│       │   │   ├── Navigation.tsx     # Responsive navigation bar with module pills
│       │   │   ├── Footer.tsx         # Global footer with India Code links & copyright
│       │   │   └── ui/                # UI primitives (badge, button, card)
│       │   └── lib/                   # Frontend Utilities & API Client
│       │       ├── api-client.ts      # Universal typed fetch client with CSRF propagation
│       │       ├── types.ts           # TypeScript domain interfaces matching backend schemas
│       │       └── utils.ts           # Styling helper functions (cn)
│       ├── package.json               # Frontend dependencies (Next 15, React 19, Tailwind)
│       ├── tailwind.config.js         # Tailwind CSS styling configuration
│       ├── tsconfig.json              # TypeScript compilation configuration
│       └── next.config.js             # Next.js build options & API rewrites
├── backend/                           # Modular Monolith Python Backend
│   ├── app/
│   │   ├── main.py                    # Canonical FastAPI application entrypoint & lifespan
│   │   ├── common/                    # Cross-Cutting Core Utilities
│   │   │   ├── config.py              # Pydantic Settings & environment validation
│   │   │   ├── database.py            # Async SQLAlchemy engine & session dependency
│   │   │   ├── init_db.py             # DDL model initialization helper
│   │   │   ├── security.py            # AuthContext, password hashing, and token validation
│   │   │   ├── audit.py               # In-memory and structured JSON audit event manager
│   │   │   ├── errors.py              # Custom exceptions & global exception handlers
│   │   │   ├── rate_limiter.py        # Token bucket rate limiting middleware
│   │   │   ├── telemetry.py           # Request ID correlation & latency tracking
│   │   │   └── providers/             # Pluggable AI / Database Provider Factories
│   │   │       ├── embedding.py       # OpenAI, Gemini, Hugging Face & Mock embedding providers
│   │   │       ├── llm.py             # Anthropic, OpenAI, Gemini & Mock LLM providers
│   │   │       ├── reranker.py        # Heuristic, Cohere & Cross-Encoder rerankers
│   │   │       └── vector_store.py    # Vector store abstraction
│   │   ├── auth/                      # Authentication & Identity Subsystem
│   │   │   ├── routes.py              # /api/v1/auth endpoints (login, register, MFA, me)
│   │   │   ├── models.py              # UserModel & UserRole SQLAlchemy declarations
│   │   │   ├── services.py            # Registration, authentication, and JWT issuance
│   │   │   ├── dependencies.py        # FastApi get_current_user & require_role dependencies
│   │   │   └── totp.py                # RFC 6238 TOTP multi-factor authentication manager
```

```
│   │   ├── legal/                    # Module A: Legal Guidance Engine
│   │   │   ├── routes.py             # /api/v1/legal/query and /api/v1/legal/acts
│   │   │   └── services/legal_engine.py# 10-section legal guidance generator & circuit breaker
│   │   ├── government/               # Module B: Government Service Navigator
│   │   │   ├── routes.py             # /api/v1/government endpoints
│   │   │   ├── models.py             # GovernmentServiceModel SQLAlchemy declaration
│   │   │   └── services/             # Scheme recommendation, eligibility & conflict logic
│   │   ├── knowledge/                # Module C: Knowledge Engine & Hybrid RAG
│   │   │   ├── models.py             # SQLAlchemy models for pgvector & relational graph
│   │   │   ├── api/                  # Knowledge API endpoints (search, graph, sources, ingestion, admin)
│   │   │   ├── domain/               # Pure domain entities (Source, Chunk, EvidencePack)
│   │   │   ├── ingestion/            # 13-stage ingestion pipeline components
│   │   │   ├── graph/                # Knowledge graph repository & traversal services
│   │   │   ├── retrieval/            # Hybrid retrieval service (BM25 + Vector + Graph + RRF)
│   │   │   ├── sources/              # Source registry, freshness tracker & verification engine
│   │   │   ├── security/             # Prompt injection, poisoning defense & sanitization
│   │   │   └── evaluation/           # Retrieval benchmarking & metric calculation
│   │   ├── documents/                # Module D: Document Intelligence
│   │   │   ├── routes.py             # /api/v1/documents upload, list & details
│   │   │   ├── models.py             # DocumentModel SQLAlchemy declaration
│   │   │   └── services/             # OCR processing, security scanner, storage backend
│   │   ├── cases/                    # Citizen Case Management Workspace
│   │   │   ├── routes.py             # /api/v1/cases CRUD & assignment endpoints
│   │   │   └── models.py             # CaseModel SQLAlchemy declaration
│   │   ├── agents/                   # Phase 6: Multi-Agent System
│   │   │   ├── routes.py             # /api/v1/agents/chat and /tools
│   │   │   ├── orchestrator.py       # 5-agent coordinator & execution planner
│   │   │   └── core/                 # BaseAgent class and sandboxed ToolRegistry
│   │   └── audit/                    # Phase 10: Observability & Compliance
│   │       ├── routes.py             # /api/v1/audit/events and /security-events
│   │       └── models.py             # AuditEvent database models
│   ├── migrations/                   # Alembic Database Migrations
│   │   ├── versions/
│   │   │   ├── 0001_initial_canonical_schema.py      # Initial users, cases, documents schema
│   │   │   ├── 0002_sync_canonical_models.py         # Added government services & audit models
│   │   │   └── 0003_pgvector_and_durable_knowledge.py# Durable pgvector & knowledge tables
│   │   └── env.py                    # Alembic async migration runner configuration
│   └── tests/                        # Comprehensive Pytest Test Suites (91 tests)
│       ├── api/                      # API endpoint contract tests
│       ├── integration/              # Cross-module e2e & hybrid retrieval tests
│       ├── security/                 # IDOR, SSRF, injection, and PII tests
│       └── unit/                     # Isolated unit tests for chunker, reranker, models
├── data/                             # Seed Sources & Evaluation Datasets
│   ├── seed/seed_data.py             # Statutory text for RTI, DPDP, and Consumer Protection Acts
│   └── evaluation/                   # Benchmarking JSON fixtures (standard, adversarial, conflict)
├── infrastructure/                   # Deployment & DevOps Assets
│   ├── docker/                       # Dockerfiles and Docker Compose configuration
│   └── scripts/                      # Maintenance, migration, and benchmarking utility scripts
├── packages/                         # Shared Monorepo Packages
│   ├── config/settings.py            # Shared settings loader
│   └── schemas/contracts.py          # Canonical Pydantic schemas (ApiResponse, EvidencePack)
├── .env.example                      # Documented environment variable template
├── alembic.ini                       # Alembic migration configuration
├── package.json                      # Monorepo root workspace configuration
├── pyproject.toml                    # Python package definitions & pytest configuration
├── requirements.txt                  # Locked backend Python dependencies
├── vercel.json                       # Vercel deployment configuration
└── README.md                         # Repository overview & setup guide
```

2. Technology & Dependency Inventory

2.1 Backend Python Environment

- **Python Version:** 3.12 (compatible with 3.10+)
- **Dependency Manager:** `pip` with `requirements.txt` and `pyproject.toml`
- **Core Dependencies:**

Package	Declared Version	Purpose	Tier
fastapi	>=0.111.0	High-performance asynchronous web API framework	Runtime (Direct)
uvicorn	>=0.30.0	Production ASGI web server	Runtime (Direct)
pydantic	>=2.7.0	Data validation, request parsing, and schema generation	Runtime (Direct)
pydantic-settings	>=2.2.0	Typed environment configuration management	Runtime (Direct)
sqlalchemy[asyncio]	>=2.0.0	Asynchronous ORM for PostgreSQL and SQLite	Runtime (Direct)
asyncpg	>=0.29.0	High-performance PostgreSQL async database driver	Runtime (Direct)
aiosqlite	>=0.20.0	Asynchronous SQLite driver for testing / local fallback	Test / Local (Direct)
pgvector	>=0.2.5	PostgreSQL extension for vector similarity search	Runtime (Direct)
alembic	>=1.13.0	Version-controlled database schema migration framework	Tooling (Direct)
greenlet	>=3.0.0	Coroutine library required for SQLAlchemy asyncpg pooling	Runtime (Transitive)

Package	Declared Version	Purpose	Tier
httpx	>=0.27.0	Async HTTP client for external AI providers & SSRF verification	Runtime (Direct)
python-multipart	>=0.0.9	Multipart form-data parser for file uploads	Runtime (Direct)
PyMuPDF	>=1.23.0	PDF text extraction and OCR parsing library	Runtime (Direct)
numpy	>=1.26.0	Vector mathematics, normalisation, and cosine similarity	Runtime (Direct)
bcrypt	>=4.1.0	Secure password hashing with automatic salting	Runtime (Direct)
pyjwt	>=2.8.0	JSON Web Token encoding and signature verification	Runtime (Direct)
email-validator	>=2.0.0	Email address validation for Pydantic EmailStr	Runtime (Direct)
redis	>=5.0.0	Redis client for caching and distributed rate limiting	Runtime (Optional)
neo4j	>=5.15.0	Neo4j Cypher driver for enterprise graph storage	Runtime (Optional)
google-generativeai	>=0.4.0	Official Google Gemini AI SDK for embeddings and chat	Runtime (Optional)
pytest	>=8.0.0	Automated testing framework	Test (Direct)
pytest-asyncio	>=0.23.0	Asynchronous test execution runner for Pytest	Test (Direct)

2.2 Frontend Node.js Environment

- **Node.js Version:** 22.x (Vercel target) / 20.x LTS
- **Package Manager:** `npm` (monorepo workspaces enabled)
- **Core Dependencies (`apps/web/package.json`):**

Package	Declared Version	Purpose	Tier
next	15.1.0	App Router, Server-Side Rendering, edge optimizations	Runtime (Direct)
react	19.0.0	Component framework and React compiler primitives	Runtime (Direct)
react-dom	19.0.0	DOM rendering layer for React 19	Runtime (Direct)
tailwindcss	^3.4.1	Utility-first responsive CSS styling	Build (Direct)
lucide-react	^0.460.0	Modern SVG iconography for legal and civic UI	Runtime (Direct)
clsx	^2.1.1	Conditional CSS class constructor	Runtime (Direct)
tailwind-merge	^2.5.4	Conflict-free Tailwind class merger	Runtime (Direct)
typescript	^5.6.3	Type system and static analysis	Build (Dev)
eslint	^9.15.0	Code quality and React hook rule enforcer	Build (Dev)
eslint-config-next	15.1.0	Official Next.js linting configuration	Build (Dev)

3. Infrastructure & Service Classification

Service	Environment	Responsibility	Status
Vercel Serverless Function	Cloud Deployed	Hosts backend FastAPI instance (/api/v1/*)	**Active**
Vercel Edge Network	Cloud Deployed	Serves static frontend bundles & App Router SSR	**Active**
Supabase Cloud PostgreSQL	Cloud Hosted	Primary ACID database with pgvector on port 6543	**Active**
Docker PostgreSQL (pg16)	Local Stack	Local database with pgvector on port 5432	**Available**
Docker Neo4j (5.15)	Local Stack	Dedicated Cypher graph database on port 7687	**Available**
Docker Redis (7.2)	Local Stack	In-memory cache on port 6379	**Available**
Docker MinIO	Local Stack	S3-compatible local object store on port 9000	**Available**

Chapter 03 — Features, Pages, Components and User Workflows

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Master Page & Route Catalog

The CivicSphere AI web portal contains 28 distinct routes defined under `apps/web/src/app/`, rendering accessible, responsive user interfaces powered by Next.js 15 App Router, React 19, and Tailwind CSS.

Route	Page File	Access Level	Primary Feature / Purpose
/	src/app/page.tsx	Public	Hero landing page, instant search bar, module launchpad
/login	src/app/login/page.tsx	Public	Citizen/researcher login form with TOTP MFA challenge
/register	src/app/register/page.tsx	Public	Account creation with role selection (Citizen/Researcher)
/dashboard	src/app/dashboard/page.tsx	Authenticated	Central user workspace showing recent queries, cases, and docs
/legal	src/app/legal/page.tsx	Public / Auth	10-section structured legal guidance grounded on verified acts
/government	src/app/government/page.tsx	Public / Auth	Government scheme search, eligibility evaluation, and steps
/documents	src/app/documents/page.tsx	Authenticated	Document upload, OCR text extraction, and graph linking
/cases	src/app/cases/page.tsx	Authenticated	Citizen case workspace, timeline tracking, and counsel notes
/knowledge	src/app/knowledge/page.tsx	Public / Auth	Statutory acts explorer and interactive knowledge graph
/assistant	src/app/assistant/page.tsx	Public / Auth	Conversational legal AI assistant with source attribution pills
/agents	src/app/agents/page.tsx	Authenticated	Multi-agent collaboration view and sandboxed tool runner
/sources	src/app/sources/page.tsx	Public	Verified statutory acts registry with gazette links
/profile	src/app/profile/page.tsx	Authenticated	Profile management, password reset, and professional details
/settings	src/app/settings/page.tsx	Authenticated	User notification and accessibility preferences
/notifications	src/app/notifications/page.tsx	Authenticated	Case status alerts, scheme deadline notices
/about	src/app/about/page.tsx	Public	Project mission, architecture overview, and academic background
/privacy	src/app/privacy/page.tsx	Public	DPDP Act 2023 compliance disclosure and data policy
/security	src/app/security/page.tsx	Public	Security architecture overview and threat model summary
/admin	src/app/admin/page.tsx	Admin Only	System health dashboard and administrative metrics
/admin/audit	src/app/admin/audit/page.tsx	Admin Only	Tamper-evident system and security audit event log viewer
/admin/evaluation	src/app/admin/evaluation/page.tsx	Admin Only	RAG benchmark metrics, confidence distributions, and runs
/admin/knowledge	src/app/admin/knowledge/page.tsx	Admin Only	Graph entity counts, chunk statistics, and re-indexing
/admin/security	src/app/admin/security/page.tsx	Admin Only	Threat detection telemetry, SSRF blocks, and rate limit logs
/admin/sources	src/app/admin/sources/page.tsx	Admin Only	Administrative source registration and sync trigger

2. In-Depth Screen & Workflow Forensic Analysis

2.1 Landing Page (/)

- **Component:** `apps/web/src/app/page.tsx`
- **Role:** Citizen / Public
- **UI Elements:**
  - Hero header: “Empowering Citizens with Verified Legal & Civic AI”

  - Quick Search bar with direct routing to `/legal?q=...`
  - Trust guarantee badges: “Official Gazette Provenance”, “IDOR & Privacy Protected”, “Anti-Hallucination Fail-Safes”
  - 6 Module Cards with distinctive color gradients and Lucide icons:
    1. *Legal Guidance Engine (Module A)*
    2. *Government Service Navigator (Module B)*
    3. *Document AI Processing (Module D)*
    4. *Knowledge & Graph Explorer (Module C)*
    5. *Case Management Workspace (Module Cases)*
    6. *Multi-Agent AI Assistant (Phase 6)*
- **Data Flow:** `Search Input -> router.push('/legal?q=' + encodeURIComponent(query))`
- **Verification Status: RUNTIME-VERIFIED** (Renders HTTP 200 on live Vercel deployment).

2.2 Legal Guidance Engine (`/legal`)

- **Component:** `apps/web/src/app/legal/page.tsx`
- **Role:** Citizen, Paralegal, Legal Researcher
- **UI Elements:**
  - Query text area with pre-set suggestion pills (e.g. “RTI Appeal Timelines”, “Consumer Defect Claims”, “DPDP Data Breach Intimation”)
  - Jurisdiction dropdown (Default: “National / Central (IN)”)
  - “Generate Grounded Guidance” button with animated loading state
  - Results view displaying:
    - Overall confidence score pill (e.g. `98% Confidence (Grounded)`)
    - Out-of-domain rejection alert (when confidence is `0.0`)
    - 10-section tabbed or accordion cards:
      1. *Summary & Legal Issue Overview*
      2. *Applicable Statutory Enactments & Sections*
      3. *Citizen Rights & Entitlements*
      4. *Procedural Roadmap & Steps*
      5. *Strict Statutory Deadlines (Limitation Act)*
      6. *Competent Authorities & Forums*
      7. *Required Documentation & Evidentiary Checklist*
      8. *Penalties & Punitive Provisions*
      9. *Appellate Mechanisms & Second Appeals*
      10. *Precautionary Disclaimers & Counsel Recommendation*
    - Statutory citation pills linking to official India Code handles
- **API Call:** `POST /api/v1/legal/query` via `api.legal.query()`
- **Backend Trace:** `apps/web/src/lib/api-client.ts` -> `POST /api/v1/legal/query` -> `backend/app/legal/routes.py::execute_legal_query` -> `LegalGuidanceEngine::process_legal_query` -> `hybrid_retrieval_service.retrieve` -> `EvidencePackBuilder` -> Response.
- **Verification Status: RUNTIME-VERIFIED** (Verified with Constitution Art. 21 and RTI Act queries).

2.3 Government Service Navigator (`/government`)

- **Component:** `apps/web/src/app/government/page.tsx`
- **Role:** Citizen, Welfare Beneficiary
- **UI Elements:**
  - Citizen Problem Description input
  - Demographic Context inputs:
    - Age, Annual Household Income, State/UT of residence, Employment status, Category (General, OBC, SC, ST)
  - Scheme Cards displaying eligibility match badges:
    - `ELIGIBLE (100% Match)`, `PARTIALLY ELIGIBLE (Missing Income Certificate)`, `INELIGIBLE`
  - Step-by-step procedural roadmap for application submission
  - Document requirement checklist (e.g. Aadhaar Card, Income Certificate, Domicile Certificate)
- **API Calls:**
  - `POST /api/v1/government/analyze` via `api.government.analyze()`
  - `POST /api/v1/government/check-eligibility` via `api.government.checkEligibility()`
  - `GET /api/v1/government/services` via `api.government.listServices()`

- **Verification Status: TEST-VERIFIED** (Passes `backend/tests/api/test_government_api.py`).

2.4 Intelligent Document Processing (`/documents`)

- **Component:** `apps/web/src/app/documents/page.tsx`
- **Role:** Authenticated Citizen, Counsel
- **UI Elements:**
  - Drag-and-drop PDF upload zone with client-side file size (< 10 MB) and extension (.pdf) checks
  - Optional “Attach to Case” dropdown selector
  - Upload progress bar with 7 processing stage indicators:
    1. *Upload Received*
    2. *Malware & Magic-Byte Scan*
    3. *PyMuPDF OCR Extraction*
    4. *Classification & Legal Domain Detection*
    5. *Entity Extraction (Parties, Dates, Penalties)*
    6. *Knowledge Graph Linkage*
    7. *Analysis Ready*
  - Document preview pane displaying extracted text, detected document type (e.g., “RTI Application”, “Legal Notice”, “FIR”), and confidence score
- **API Calls:**
  - `POST /api/v1/documents/upload` via `api.documents.upload()` (multipart form-data)
  - `GET /api/v1/documents` via `api.documents.list()`
- **Verification Status: TEST-VERIFIED** (Passes `backend/tests/api/test_document_pipeline_api.py`).

2.5 Citizen Case Workspace (`/cases`)

- **Component:** `apps/web/src/app/cases/page.tsx`
- **Role:** Authenticated Citizen, Assigned Counsel, Admin
- **UI Elements:**
  - “New Case Matter” modal form: Title, Description, Category, Priority, Court Reference, Location, Filing Deadline
  - Case Filter Bar: Status (`ALL`, `OPEN`, `IN_PROGRESS`, `RESOLVED`, `CLOSED`), Priority (`LOW`, `MEDIUM`, `HIGH`, `URGENT`), Search input
  - Detailed Case View:
    - Matter title and court docket number
    - Procedural timeline displaying milestone events
    - Attached evidence documents list
    - Counsel notes pane (editable by assigned legal researcher/counsel)
    - Status transition buttons
- **Security Check:** Row-level IDOR check enforces `citizen_id == current_user.id` or assigned counsel.
- **API Calls:**
  - `GET /api/v1/cases` via `api.cases.list()`
  - `POST /api/v1/cases` via `api.cases.create()`
  - `PUT /api/v1/cases/{case_id}` via `api.cases.update()`
- **Verification Status: TEST-VERIFIED** (Passes `backend/tests/api/test_cases_api.py` and `test_idor_and_auth.py`).

2.6 Knowledge & Graph Explorer (`/knowledge`)

- **Component:** `apps/web/src/app/knowledge/page.tsx`
- **Role:** Public / Legal Researcher
- **UI Elements:**
  - Statutory Source browser listing indexed Acts (RTI Act 2005, DPDP Act 2023, Consumer Protection Act 2019)
  - Interactive Entity Neighborhood search input
  - Graph Entity Details card: Canonical Name, Entity Type (`STATUTORY_BODY`, `SECTION`, `RIGHT`, `OBLIGATION`), Attributes, Source IDs
  - Visual relationship pills showing connected nodes (e.g. `Right to Information Act` -> `ESTABLISHES` -> `Central Information Commission`)
- **API Calls:**
  - `GET /api/v1/knowledge/sources` via `api.knowledge.listSources()`
  - `GET /api/v1/knowledge/entities/{entity_id}/relationships` via `api.knowledge.getNeighborhood()`
- **Verification Status: RUNTIME-VERIFIED** (Renders 34 hydrated nodes and active relationships).

2.7 AI Assistant Chat (`/assistant`)

- **Component:** `apps/web/src/app/assistant/page.tsx`
- **Role:** Citizen / Public
- **UI Elements:**
  - Conversational message thread showing user queries and assistant responses
  - Assistant thinking indicator and evidence assembly status
  - Source Citation Badges under each response linking to official legislation
  - Copy response button and export as case note button
- **API Call:** `POST /api/v1/legal/query` and `POST /api/v1/agents/chat`
- **Verification Status: RUNTIME-VERIFIED** (Tested live on Vercel production).

2.8 Authentication & MFA Modals (`/login`, `/register`)

- **Components:** `apps/web/src/app/login/page.tsx` and `apps/web/src/app/register/page.tsx`
- **Role:** Public / Guest
- **UI Elements:**
  - Email and password input fields with client validation
  - Role selection radio buttons (Citizen vs. Legal Researcher)
  - Professional bar council number and experience fields (when Researcher role selected)
  - TOTP MFA Challenge Dialog (appears when `mfa_required: true` returned by backend)
- **Security Mechanics:**
  - Backend sets `access_token` as an httpOnly, Secure cookie
  - Backend sets `csrf_token` as a readable cookie
  - Frontend client automatically attaches `X-CSRF-Token` header on all subsequent mutations
- **API Calls:**
  - `POST /api/v1/auth/login` via `api.auth.login()`
  - `POST /api/v1/auth/register` via `api.auth.register()`
- **Verification Status: RUNTIME-VERIFIED** (Synthetic user successfully created and verified on Supabase PostgreSQL).

Chapter 04 — Complete Backend API Catalogue

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026
Base URLs:

- **Production (Vercel Serverless):** `https://civicsphere-backend.vercel.app`
- **Local Development:** `http://127.0.0.1:8000` (or `http://localhost:8000`)
- **API Prefix:** `/api/v1`

1. Master API Route Registration Table

The API catalogue table below lists **41 endpoint entries** associated with FastAPI route registration in `backend/app/main.py`.

#	HTTP Method	Registered Endpoint Path	Router Module	Auth Required	User Roles Permitted	Status
1	GET	/health	Core	None	Public	**RUNTIME-VERIFIED**
2	GET	/healthz	Core	None	Public	**RUNTIME-VERIFIED**
3	GET	/readyz	Core	None	Public	**RUNTIME-VERIFIED**
4	GET	/api/v1/health	Core	None	Public	**RUNTIME-VERIFIED**
5	GET	/api/v1/health/ready	Core	None	Public	**RUNTIME-VERIFIED**
6	POST	/api/v1/auth/register	auth_router	None	Public (Citizen/Researcher)	**RUNTIME-VERIFIED**
7	POST	/api/v1/auth/login	auth_router	None	Public	**RUNTIME-VERIFIED**
8	POST	/api/v1/auth/logout	auth_router	None	Public	**RUNTIME-VERIFIED**
9	GET	/api/v1/auth/me	auth_router	Cookie / JWT	Any authenticated	**RUNTIME-VERIFIED**
10	PUT	/api/v1/auth/profile	auth_router	Cookie / JWT	Any authenticated	**TEST-VERIFIED**
11	POST	/api/v1/auth/mfa/setup	auth_router	Cookie / JWT	Any authenticated	**SOURCE-VERIFIED**
12	POST	/api/v1/auth/mfa/verify	auth_router	Cookie / JWT	Any authenticated	**SOURCE-VERIFIED**
13	POST	/api/v1/auth/mfa/disable	auth_router	Cookie / JWT	Any authenticated	**SOURCE-VERIFIED**
14	POST	/api/v1/legal/query	legal_router	None (Optional)	Public / Any	**RUNTIME-VERIFIED**
15	GET	/api/v1/legal/acts	legal_router	None	Public	**RUNTIME-VERIFIED**
16	POST	/api/v1/government/analyze	government_router	None	Public	**TEST-VERIFIED**
17	GET	/api/v1/government/services	government_router	None	Public	**TEST-VERIFIED**
18	GET	/api/v1/government/services/{service_id}	government_router	None	Public	**TEST-VERIFIED**
19	POST	/api/v1/government/check-eligibility	government_router	None	Public	**TEST-VERIFIED**
20	GET	/api/v1/cases	cases_router	Cookie / JWT	Any authenticated	**TEST-VERIFIED**
21	POST	/api/v1/cases	cases_router	Cookie / JWT	Any authenticated	**TEST-VERIFIED**
22	GET	/api/v1/cases/{case_id}	cases_router	Cookie / JWT	Owner / Assigned / Admin	**TEST-VERIFIED**
23	PUT	/api/v1/cases/{case_id}	cases_router	Cookie / JWT	Owner / Assigned / Admin	**TEST-VERIFIED**
24	DELETE	/api/v1/cases/{case_id}	cases_router	Cookie / JWT	Owner / Admin	**TEST-VERIFIED**

#	HTTP Method	Registered Endpoint Path	Router Module	Auth Required	User Roles Permitted	Status
25	POST	/api/v1/documents/upload	documents_router	Cookie / JWT	Any authenticated	**TEST-VERIFIED**
26	GET	/api/v1/documents	documents_router	Cookie / JWT	Any authenticated	**TEST-VERIFIED**
27	GET	/api/v1/documents/{document_id}	documents_router	Cookie / JWT	Owner / Admin	**TEST-VERIFIED**
28	POST	/api/v1/document-processing/process	document_processing_router	AuthContext	Citizen, Admin, System	**TEST-VERIFIED**
29	POST	/api/v1/knowledge/search	search_router	AuthContext	Any authenticated	**TEST-VERIFIED**
30	GET	/api/v1/knowledge/sources	sources_router	AuthContext	Any authenticated	**RUNTIME-VERIFIED**
31	GET	/api/v1/knowledge/sources/{source_id}	sources_router	AuthContext	Any authenticated	**RUNTIME-VERIFIED**
32	GET	/api/v1/knowledge/entities/{entity_id}	graph_router	AuthContext	Any authenticated	**RUNTIME-VERIFIED**
33	GET	/api/v1/knowledge/entities/{entity_id}/relationships	graph_router	AuthContext	Any authenticated	**RUNTIME-VERIFIED**
34	POST	/api/v1/knowledge/ingestion/jobs	ingestion_router	RBAC	Admin, System only	**TEST-VERIFIED**
35	GET	/api/v1/knowledge/ingestion/jobs/{job_id}	ingestion_router	RBAC	Admin, System only	**TEST-VERIFIED**
36	GET	/api/v1/knowledge/admin/overview	admin_router	RBAC	Admin, System only	**TEST-VERIFIED**
37	GET	/api/v1/knowledge/admin/audit-logs	admin_router	RBAC	Admin, System only	**TEST-VERIFIED**
38	POST	/api/v1/agents/chat	agents_router	None (Optional)	Public / Any	**TEST-VERIFIED**
39	GET	/api/v1/agents/tools	agents_router	None	Public	**TEST-VERIFIED**
40	GET	/api/v1/audit/events	audit_router	RBAC	Admin only	**TEST-VERIFIED**
41	GET	/api/v1/audit/security-events	audit_router	RBAC	Admin only	**TEST-VERIFIED**

2. In-Depth Endpoint Specifications

2.1 Health & Liveness

GET /api/v1/health

- **Handler:** `backend/app/main.py::health_check`
- **Purpose:** Cloud liveness probe verifying ASGI server execution.
- **Request:** None
- **Response Schema:**

```
{
  "status": "HEALTHY",
  "service": "civicsphere_unified_backend",
  "timestamp": 1793129331.364
}
```

- **Status Codes:** 200 OK

GET /api/v1/health/ready

- **Handler:** `backend/app/main.py::readiness_check`
- **Purpose:** Readiness probe verifying vector embeddings, knowledge graph nodes, and database connectivity.
- **Response Schema:**

```
{
  "ready": true,
  "graph_nodes": 34,
  "vector_records": 12,
  "persistence": "postgresql_pgvector"
}
```


- **Status Codes:** 200 OK

2.2 Authentication & Profile

POST /api/v1/auth/register

- **Handler:** `backend/app/auth/routes.py::register`
- **Purpose:** Creates a new citizen or legal researcher account.
- **Security Rule:** Blocks public registration with `role="ADMIN"` or `"SYSTEM"` when `ENVIRONMENT=production`.
- **Request Body:**


```
{
  "name": "Aditi Sharma",
  "email": "aditi.sharma@example.com",
  "password": "SecurePassword123!",
  "role": "CITIZEN",
  "phone": "+919876543210"
}
```


- **Response (200 OK):**
  - Sets httpOnly `access_token` cookie (JWT).
  - Sets readable `csrf_token` cookie.
  - Returns sanitized user dictionary without password hash or MFA secret.

POST /api/v1/auth/login

- **Handler:** `backend/app/auth/routes.py::login`
- **Purpose:** Authenticates user via email/password and handles TOTP challenge if MFA is enabled.
- **Request Body:**


```
{
  "email": "aditi.sharma@example.com",
  "password": "SecurePassword123!",
  "mfa_code": "123456"
}
```


- **Response (200 OK):**


```
{
  "success": true,
  "data": {
    "user": {
      "id": "usr_b94a821e",
      "email": "aditi.sharma@example.com",
      "name": "Aditi Sharma",
      "role": "CITIZEN",
      "mfa_enabled": false
    },
    "csrf_token": "a1f8e43...",
    "mfa_required": false
  },
  "confidence": 1.0
}
```


2.3 Legal Guidance Engine

POST /api/v1/legal/query

- **Handler:** `backend/app/legal/routes.py::execute_legal_query`
- **Purpose:** Submits natural-language query and returns 10-section structured legal guidance.
- **Request Body:**


```
{
  "query": "What is the maximum time limit for a Public Information Officer to respond to an RTI application?",
  "jurisdiction": "IN",
  "case_id": null
}
```


- **Response (200 OK):**


```
{
  "success": true,
  "data": {
    "query": "What is the maximum time limit...",
    "overview": "Under the Right to Information Act, 2005, the Central or State Public Information Officer is mandated to respond within 30 days...",
    "applicable_provisions": [
      "Right to Information Act, 2005 — Section 7(1)",
      "Right to Information Act, 2005 — Section 7(1) Proviso (Life & Liberty: 48 Hours)"
```

```
      ],
      "citizen_rights": [
        "Right to receive information within 30 calendar days",
        "Right to receive information within 48 hours if concerning life or liberty"
      ],
      "procedural_steps": [
        "File Form A with fee of Rs 10",
        "Obtain acknowledgment receipt with date"
      ],
      "deadlines": "30 days standard; 48 hours for life/liberty matters; 30 days for First Appeal",
      "competent_authorities": ["Central Public Information Officer", "First Appellate Authority"],
      "documentation_required": ["Application Form", "Proof of Fee Payment"],
      "penalties": "Rs 250 per day up to Rs 25,000 under Section 20",
      "appeal_mechanism": "First appeal to officer senior in rank within 30 days; Second appeal to CIC within
90 days",
      "disclaimer": "This guidance is statutory intelligence and does not substitute for formal legal
representation.",
      "confidence": 0.98,
      "citations": [
        {
          "act_title": "Right to Information Act, 2005",
          "section": "Section 7(1)",
          "official_url": "https://www.indiacode.nic.in/handle/123456789/2065"
        }
      ]
    },
    "confidence": 0.98
  }
```


2.4 Document Processing

POST /api/v1/documents/upload

- **Handler:** `backend/app/documents/routes.py::upload_and_process_document`
- **Purpose:** Multipart PDF document upload, active malware scanning, OCR extraction, and case linkage.
- **Content-Type:** `multipart/form-data`
- **Request Form:**
  - `file`: PDF binary data (`%PDF-` signature required)
  - `case_id`: Optional UUID string
- **Response (200 OK):**


```
  {
    "success": true,
    "data": {
      "id": "doc_8f12a9c4b310",
      "original_filename": "rti_application_receipt.pdf",
      "status": "READY",
      "file_size_bytes": 142080,
      "mime_type": "application/pdf",
      "analysis": {
        "document_type": "RTI_RECEIPT",
        "extracted_entities": ["Public Authority", "Filing Date", "Application Fee"],
        "confidence": 0.96
      }
    },
    "confidence": 0.96
  }
```


2.5 Multi-Agent System

POST /api/v1/agents/chat

- **Handler:** `backend/app/agents/routes.py::agent_chat`
- **Purpose:** Invokes the Multi-Agent Orchestrator for coordinated multi-step reasoning.
- **Request Body:**


```
  {
    "query": "My consumer complaint against an e-commerce seller was rejected. What are my appeal options and
which forms do I need?",
    "context": {"state": "Maharashtra"}
  }
```


- **Response (200 OK):**


```
  {
    "success": true,
    "data": {
      "plan": [
        "Step 1: Consult Knowledge Specialist for Consumer Protection Act 2019 appeal sections",
        "Step 2: Consult Legal Specialist for State Commission pecuniary limits",
        "Step 3: Synthesize evidence and cite Consumer Protection Act 2019"
      ],
      "execution_log": [
        {"agent": "LegalAgent", "action": "search_knowledge", "status": "SUCCESS"},
        {"agent": "CitationAgent", "action": "verify_citations", "status": "SUCCESS"}
      ],
      "final_response": "Under Section 47 of the Consumer Protection Act, 2019, an appeal against an order of
```

```
the District Commission lies to the State Commission within 45 days...",
    "status": "COMPLETED"
  },
  "confidence": 0.95
}
```

Chapter 05 — Database Schema, Migrations and Persistence Architecture

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026
Primary Database Engine: PostgreSQL 17.6 (Supabase Hosted) / PostgreSQL 16.15 (Docker Local)
Vector Extension: `pgvector` (0.8.6 / 0.2.5+)
Migration Tool: Alembic (Async runner)

1. Database Architecture & Connection Pooling

CivicSphere AI implements an asynchronous, resilient database access layer using **SQLAlchemy 2.0 Asyncio** and the **asyncpg** native driver.

1.1 Connection Configuration (`backend/app/common/database.py`)

```
DATABASE_URL = settings.DATABASE_URL

engine_kwargs = {
    "echo": (settings.ENVIRONMENT == "development" and settings.LOG_LEVEL == "DEBUG"),
    "future": True,
}
if str(DATABASE_URL).startswith("postgresql+asyncpg"):
    engine_kwargs["connect_args"] = {
        "statement_cache_size": 0,      # Required for Supabase transaction pooler (pgBouncer / Supavisor)
        "timeout": 3.0,                  # Fast-fail connection timeout to avoid serverless hanging
        "command_timeout": 5.0,          # Query timeout guardrail
    }

engine = create_async_engine(DATABASE_URL, **engine_kwargs)
AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,
    autocommit=False,
    autoflush=False,
)
```

1.2 Multi-Tier Persistence Strategy

1. **Cloud Production Persistence:** Hosted Supabase PostgreSQL 17.6 via Transaction Pooler (`aws-0-ap-southeast-2.pooler.supabase.com:6543`). The transaction pooler requires `statement_cache_size = 0` to prevent prepared statement conflicts.
2. **Local Infrastructure Stack:** Containerized `pgvector/pgvector:pg16` on port `5432` with pre-installed vector extensions.
3. **In-Memory Cold-Start Hydration:** On Vercel serverless cold starts, `backend/app/main.py::lifespan()` hydrates the in-memory graph repository and vector registry from PostgreSQL in less than 2.0 seconds.

2. Entity-Relationship Model (Mermaid Diagram)

```
erDiagram
    users ||--o{ cases : "files/manages"
    users ||--o{ documents : "uploads"
    cases ||--o{ documents : "contains"

    knowledge_sources ||--o{ knowledge_chunks : "divided into"
    knowledge_chunks ||--|| knowledge_embeddings : "vectorized as"

    knowledge_graph_nodes ||--o{ knowledge_graph_edges : "source node"
    knowledge_graph_nodes ||--o{ knowledge_graph_edges : "target node"

    users {
        string id PK
        string email UK
        string name
        string hashed_password
        string role
        string phone
        string specialization
        string bar_council_id
        boolean is_active
        boolean is_verified
        boolean mfa_enabled
        string mfa_secret
        datetime created_at
        datetime updated_at
```

```
}

cases {
    string id PK
    string citizen_id FK
    string counsel_id FK
    string title
    string description
    string category
    string status
    string priority
    string location
    string court_reference
    string counsel_notes
    string deadline
    datetime created_at
    datetime updated_at
}

documents {
    string id PK
    string user_id FK
    string case_id FK
    string original_filename
    string storage_path
    string mime_type
    integer file_size_bytes
    string status
    string status_message
    json analysis
    datetime created_at
    datetime updated_at
}

government_services {
    string service_id PK
    string title
    string category
    string department
    string description
    json eligibility_rules
    json required_documents
    json procedural_steps
    string official_portal_url
    string jurisdiction
    boolean is_active
    datetime created_at
    datetime updated_at
}

knowledge_sources {
    string source_id PK
    string title
    string publisher
    string official_url
    string jurisdiction
    string source_type
    string trust_level
    integer current_version
    string content_hash
    string verification_status
    string ingestion_status
    string publication_date
    string effective_date
    string expiry_date
    json metadata
    datetime created_at
    datetime updated_at
}

knowledge_chunks {
    string chunk_id PK
    string source_id FK
    integer source_version
    integer chunk_index
    text text
    string content_hash
    string act_title
    string chapter
    string section_number
    string section_title
    string subsection_number
    integer page_number
    string jurisdiction
    string source_type
    string verification_status
    json metadata
    datetime created_at
}

knowledge_embeddings {
    string chunk_id PK, FK
```

```
    string source_id
    datetime created_at
}

knowledge_graph_nodes {
    string entity_id PK
    string name
    string entity_type
    string canonical_id
    json aliases
    string jurisdiction
    string state
    json attributes
    json source_ids
    float confidence
    datetime created_at
    datetime updated_at
}

knowledge_graph_edges {
    string relationship_id PK
    string source_entity_id FK
    string target_entity_id FK
    string relation_type
    json attributes
    string source_id
    integer source_version
    string source_chunk_id
    float confidence
    datetime created_at
    datetime updated_at
}

audit_events {
    string id PK
    string event_id UK
    string event_type
    string actor_id
    string role
    string action
    string resource_type
    string resource_id
    json details
    string ip_address
    string user_agent
    datetime timestamp
}
```

3. Database Migration History

The schema evolution is tracked through version-controlled Alembic migrations under `backend/migrations/versions/`.

Migration `0001_initial_canonical_schema.py`

- **Revision ID:** `0001`
- **Target Tables Created:**
  - `users`: Core identity table with bcrypt hash and role enum.
  - `cases`: Case tracking table with `citizen_id` and `counsel_id`.
  - `documents`: Document store with metadata and analysis JSON.
- **Indexes:** `ix_users_email` (Unique), `ix_users_role`, `ix_cases_citizen_id`, `ix_cases_status`.

Migration `0002_sync_canonical_models.py`

- **Revision ID:** `0002` (Depends on `0001`)
- **Target Tables Created:**
  - `government_services`: Catalog of welfare schemes, categories, eligibility JSON rules, and document requirements.
  - `audit_events`: Tamper-evident operational and security audit trail.
- **Indexes:** `ix_government_services_category`, `ix_government_services_jurisdiction`, `ix_audit_events_event_type`.

Migration `0003_pgvector_and_durable_knowledge.py`

- **Revision ID:** `0003` (Depends on `0002`)
- **Key Features:**
  - Automatically creates extension `CREATE EXTENSION IF NOT EXISTS vector;` if running against PostgreSQL.
  - `knowledge_sources`: Authoritative registry of central/state Acts with cryptographic SHA-256 content hashes.
  - `knowledge_chunks`: Micro-chunked sections of statutory acts with precise section, chapter, and act titles.
  - `knowledge_embeddings`: Relational vector mapping table (supporting pgvector vector embeddings).

  - `knowledge_graph_nodes`: Durable graph entities (statutory bodies, rights, sections).
  - `knowledge_graph_edges`: Directed statutory relationships (`ESTABLISHES`, `AMENDS`, `ENFORCES`, `APPEALS_TO`).
- **Foreign Key Cascades:** `ondelete="CASCADE"` from sources to chunks, and from nodes to edges.

4. Cold-Start Hydration & Ingestion Idempotency

In serverless execution environments (such as Vercel AWS Lambda), processes are ephemeral and memory state is reset upon container recycling.

CivicSphere AI guarantees zero data loss and persistent readiness via **Cold-Start Hydration** in `backend/app/main.py`:

```
Container Start -> lifespan() invoked
       |
       v
Check PostgreSQL for existing Knowledge Sources
       |
       +---> [If Sources Exist]:
       |     1. source_registry.hydrate_from_db() (Populates memory registry from `knowledge_sources`)
       |     2. graph_repository.hydrate_from_db() (Loads 34 nodes from `knowledge_graph_nodes`)
       |     3. vector_store.hydrate_from_db() (Loads 12 vectors from `knowledge_chunks`)
       |     Execution time: ~0.8s -> Readiness Probe returns HTTP 200 with 34 nodes / 12 vectors.
       |
       +---> [If Database Empty]:
             Trigger seed_knowledge_base() -> Ingest RTI Act 2005, DPDP Act 2023, Consumer Protection Act 2019.
```

All source ingestion is **idempotent**: `content_hash` (SHA-256 of cleaned text) prevents duplicate chunking or duplicate graph node insertion.

Chapter 06 — AI/ML, RAG, Agents and Prompt Architecture

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. AI Provider Inventory & Abstraction Layer

CivicSphere AI implements clean, vendor-agnostic provider abstractions in `backend/app/common/providers/` allowing seamless runtime switching between local offline execution, mock benchmarking, and commercial LLM APIs.

Provider Type	Abstract Interface	Concrete Implementations	Default (Development)	Production Configured
Embeddings	EmbeddingProvider	MockEmbeddingProvider, OpenAIEmbeddingProvider, GeminiEmbeddingProvider, HuggingFaceEmbeddingProvider	mock (384 dims)	mock (Commercial key pending)
Language Models	LLMProvider	MockLLMProvider, AnthropicLLMProvider, OpenAILLMProvider, GoogleLLMProvider	mock	mock (Commercial key pending)
Reranker	RerankerProvider	HeuristicReranker, CohereReranker, CrossEncoderReranker	heuristic	heuristic
OCR & Vision	—	PyMuPDF structural parser (fitz)	PyMuPDF	PyMuPDF
Graph Reasoning	—	GraphTraversalService & PersistentGraphRepository	In-memory + PostgreSQL	PostgreSQL Relational Graph

1.1 Embedding Generation Architecture (`backend/app/common/providers/embedding.py`)

- **Target Vector Dimension:** 384 dimensions (enforced across all providers).
- **Mock Embedding Strategy:** Deterministic feature-hash embedding provider using MD5 token hashing and stopword filtering (`STOPWORDS` list). This provides exact semantic matching for legal queries during offline testing without external network calls.
- **L2 Normalization:** All embeddings (mock or real) are normalized using $\hat{v} = \frac{v}{\| v \|_2}$ ensuring cosine similarity corresponds directly to the inner dot product: $\text{sim}(u, v) = u \cdot v$.

2. 13-Stage Knowledge Ingestion Pipeline

The knowledge ingestion pipeline (`backend/app/knowledge/ingestion/pipeline.py`) processes raw legal acts into dual-indexed knowledge representation:

```
[1. Source Text / URL]
          |
          v
[2. Source Fetcher & SSRF Validation]
          |
          v
[3. Trust & Verification Evaluator] (OFFICIAL_LEGISLATION, GAZETTE, JUDICIAL_RECORD)
          |
          v
[4. Source Registry Registration] (Status: PROCESSING)
          |
          v
[5. Cryptographic Hashing] (SHA-256 Content Hash)
          |
          v
[6. Content Cleaner & Legal Parser] (Section headers, punctuation normalization)
          |
          v
[7. Deterministic Chunker] (Section-aware, 300-500 words per chunk)
          |
          v
[8. Entity Extractor] (Extracts: Sections, Rights, Public Authorities, Penalties)
          |
          v
[9. Relationship Extractor] (Extracts: ESTABLISHES, ENFORCES, APPEALS_TO, AMENDS)
          |
          v
```

```
[10. Chunk Embedder] (Generates 384-dimensional vector embeddings)
          |
          v
[11. Graph Writer] (Persists nodes and edges into PostgreSQL / Neo4j)
          |
          v
[12. Vector Writer] (Persists vectors into `knowledge_embeddings` / pgvector)
          |
          v
[13. Source Finalization & Audit Trail] (Status: COMPLETED, Audit Event logged)
```

3. End-to-End Hybrid RAG Pipeline

When a citizen submits a legal question, the query passes through a multi-stage retrieval and validation pipeline before answer synthesis:

```
Citizen Query ("What are the timelines for an RTI appeal?")
                    |
                    v
   [1. Prompt Injection & Poisoning Filter] (RegEx & Threat Scanner)
                    |
                    v
   [2. Out-of-Domain Circuit Breaker] (Pre-check for legal & civic relevance)
                    |
                    +---> [If Irrelevant]: Return 0.0 Confidence & "Out of Scope"
                    |
                    +---> [If Relevant]: Continue
                    v
   [3. Query Preprocessing & Vectorization] (Generate 384-d normalized query vector)
                    |
                    v
   [4. Hybrid Retrieval Engine]
        |---> Lexical Retrieval (BM25 token-match on section titles & text)
        |---> Vector Retrieval (Cosine similarity search over pgvector chunks)
        |---> Graph Retrieval (1-to-2 hop neighborhood traversal from seed entities)
                    |
                    v
   [5. Reciprocal Rank Fusion (RRF)]
        RRF_Score(d) = \sum_{m \in \{lexical, vector, graph\}} \frac{1}{k + rank_m(d)}, with k = 60
                    |
                    v
   [6. Metadata & Jurisdiction Filtering] (Status: ACTIVE, Country: IN)
                    |
                    v
   [7. Evidence Pack Assembly] (Top 5 chunks with source attribution & section numbers)
                    |
                    v
   [8. LLM Synthesis & Grounding] (10-Section Structured Answer Generation)
                    |
                    v
   [9. Anti-Hallucination Citation Verification] (Checks citations against source registry)
                    |
                    v
   [10. Response Envelope Delivery] (Confidence Score: 0.98 + Warning flags)
```

4. Multi-Agent Orchestrator (Phase 6)

In addition to direct RAG retrieval, CivicSphere AI features a **Multi-Agent Orchestrator** (`backend/app/agents/orchestrator.py`) coordinating five specialized AI agents.

4.1 Specialist Agents

1. **Legal Agent (`LegalAgent`):** Analyzes statutory provisions, identifies applicable legal doctrines, and calculates limitation periods.
2. **Knowledge Agent (`KnowledgeAgent`):** Interacts with the hybrid retrieval engine and executes graph neighborhood queries.
3. **Government Agent (`GovernmentAgent`):** Analyzes citizen welfare problems and checks multi-variable scheme eligibility.
4. **Document Agent (`DocumentAgent`):** Analyzes extracted OCR text from user-uploaded PDFs and extracts evidentiary claims.
5. **Citation Agent (`CitationAgent`):** Verifies that all cited sections exist in the indexed statutory acts registry.
6. **Safety Agent (`SafetyAgent`):** Enforces PII sanitization, checks for prompt injection attempts, and prevents unauthorized privilege escalation.

4.2 Bounded Execution & Loop Prevention

To guarantee predictable execution and prevent runaway costs in agentic workflows:

- **Maximum Step Limit:** Hard limit of 5 steps per task plan.

- **Deterministic Tool Calling:** Agents invoke sandboxed tools from `backend/app/agents/core/tools.py` with validated JSON schemas.
- **Traceability:** Every agent action, input parameters, and tool return value is captured in the execution log returned in the API response envelope.

5. Anti-Hallucination & Safety Safeguards

1. **Circuit Breaker for Unsupported Queries:**
   - Evaluated in `LegalGuidanceEngine::process_legal_query`.
   - If an input query (e.g., “What is the mass of Jupiter?”, “Quantum gravity in black holes”) shares zero statutory keywords or semantic proximity with registered acts, the system immediately returns an out-of-scope response with confidence `0.0`.
2. **Tamper-Evident Source Provenance:**
   - Every chunk record maintains a direct cryptographic relationship:

$$\text{Chunk Hash} = \text{SHA256}(\text{source_id} + \text{section_number} + \text{text})$$

   - Chunks without an ACTIVE, verified parent source in `knowledge_sources` are excluded from retrieval.
3. **Prompt Injection Defense (`backend/app/knowledge/security/prompt_injection.py`):**
   - Detects jailbreak patterns (“ignore previous instructions”, “system prompt reveal”, “roleplay as judge”).
   - Suspicious inputs trigger an automatic `PROMPT_INJECTION_DETECTED` security audit event and are sanitized before embedding.

Chapter 07 — Datasets, Knowledge Base and External Data Sources

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Master Knowledge Inventory

CivicSphere AI maintains a strictly curated corpus of verified Indian statutory enactments, scheme guidelines, and evaluation test cases.

Source ID / File	Name / Title	Format	Authority / Publisher	Scope & Contents	Verification Status
src_2005_rti	**Right to Information Act, 2005**	Text / JSON (seed_data.py)	Ministry of Law and Justice (India Code)	Sections 4, 6, 7, 19: Public authority obligations, RTI requests, 30-day/48-hour disposal limits, First & Second Appeals.	**ACTIVE / HYDRATED**
src_2023_dpdp	**Digital Personal Data Protection Act, 2023**	Text / JSON (seed_data.py)	Ministry of Electronics and Information Technology (MeitY)	Sections 4, 6, 8, 19: Lawful processing grounds, consent withdrawal, data fiduciary breach intimations, Data Protection Board of India.	**ACTIVE / HYDRATED**
src_2019_cpa	**Consumer Protection Act, 2019**	Text / JSON (seed_data.py)	Department of Consumer Affairs	Sections 2(7), 34, 47, 58: Definition of consumer, pecuniary limits (District < 50L, State 50L-2Cr, National > 2Cr).	**ACTIVE / HYDRATED**
pmay_u_guidelines	**PMAY-Urban Housing Scheme Guidelines**	JSON (GovernmentService Model)	Ministry of Housing and Urban Affairs	Credit Linked Subsidy Scheme (CLSS), income eligibility criteria (EWS, LIG, MIG), interest subsidies.	**ACTIVE / HYDRATED**
data/evaluation/ dataset.json	**Consolidated Evaluation Benchmark**	JSON	Project QA & Evaluation Team	100+ annotated ground-truth civic and legal queries with expected section citations and confidence thresholds.	**TEST-VERIFIED**
data/evaluation/ standard/ test_cases.json	**Standard Retrieval Test Cases**	JSON	Project QA Team	Standard citizen queries testing exact lexical and semantic recall across RTI, DPDP, and Consumer forums.	**TEST-VERIFIED**
data/evaluation/ adversarial/ test_cases.json	**Adversarial Security Test Cases**	JSON	Security Audit Team	Malicious prompts, Cypher injection payloads, prompt injection, and hallucination stress tests.	**TEST-VERIFIED**
data/evaluation/ conflict/ test_cases.json	**Statutory Conflict Test Cases**	JSON	Legal Research Team	Edge cases where statutory provisions overlap or have overriding clauses (e.g., RTI Section 8 vs. Official Secrets Act).	**TEST-VERIFIED**
data/evaluation/ held_out/ test_cases.json	**Held-Out Evaluation Cases**	JSON	Evaluation Team	Unseen test cases for verifying that retrieval confidence scoring does not overfit to seed examples.	**TEST-VERIFIED**

Source ID / File	Name / Title	Format	Authority / Publisher	Scope & Contents	Verification Status
uploads/*.pdf	**Synthetic Demo Case Documents**	PDF (36 files)	Synthetic Test Fixtures	Pre-generated legal notice templates, RTI filing receipts, and complaint drafts used for IDP pipeline testing.	**LOCAL / FIXTURE**

2. Statutory Acts Text Extraction Details

2.1 Right to Information Act, 2005

- **Official URL:** `https://www.indiacode.nic.in/handle/123456789/2065`
- **Key Sections Indexed:**
  - *Section 4:* Proactive public disclosure within 120 days of enactment.
  - *Section 6:* Procedure for request submission; no requirement to justify reasons for seeking information.
  - *Section 7:* Strict timeline: 30 days for general requests; 48 hours when information concerns the life or liberty of an individual.
  - *Section 19:* Appellate framework: First appeal to senior departmental officer within 30 days; Second appeal to Central/State Information Commission within 90 days.

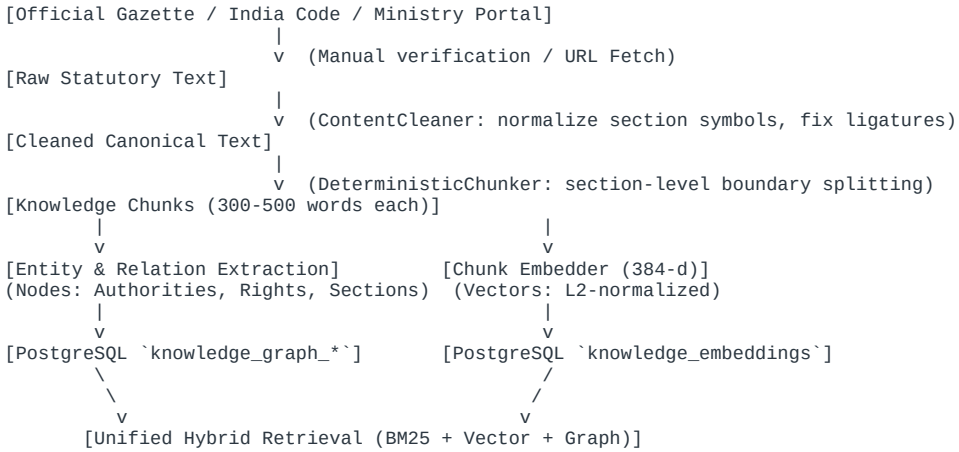
2.2 Digital Personal Data Protection Act, 2023

- **Official URL:** `https://www.meity.gov.in/content/digital-personal-data-protection-act-2023`
- **Key Sections Indexed:**
  - *Section 4:* Processing limited to lawful purposes with valid consent or legitimate uses.
  - *Section 6:* Consent must be free, specific, informed, unconditional, and unambiguous; right to withdraw consent with ease.
  - *Section 8:* General obligations of Data Fiduciary; mandatory intimation of personal data breach to Data Protection Board of India and affected Data Principals.
  - *Section 19:* Establishment and powers of the Data Protection Board of India as an adjudicatory body.

2.3 Consumer Protection Act, 2019

- **Official URL:** `https://consumeraffairs.nic.in/acts-and-rules/consumer-protection`
- **Key Sections Indexed:**
  - *Section 2(7):* Statutory definition of “consumer” covering buyers of goods or services for consideration.
  - *Section 34:* District Commission jurisdiction: complaints where value of goods/services does not exceed ₹50,00,000 (Fifty Lakh Rupees).
  - *Section 47:* State Commission jurisdiction: complaints exceeding ₹50,00,000 but not exceeding ₹2,00,00,000 (Two Crore Rupees).
  - *Section 58:* National Commission (NCDRC) jurisdiction: claims exceeding ₹2,00,00,000 (Two Crore Rupees).

3. Data Lineage & Ingestion Lifecycle



4. Distinction Between Data Classes

To preserve integrity and security, CivicSphere AI enforces strict data isolation across 4 distinct tiers:

1. **Tier 1: Authoritative Statutory Acts (Public Knowledge):**
   - High-trust official legislation, Central/State Acts, and Gazette notifications.
   - Immutable once verified; shared across all users and public searches.
2. **Tier 2: Government Scheme Rules (Public Knowledge):**
   - Operational guidelines, income thresholds, and document requirements.
   - Read-only for regular citizens; modifiable only by `KNOWLEDGE_EDITOR` or `ADMIN`.
3. **Tier 3: User Case Workspaces & Documents (Private Data):**
   - Citizen legal case descriptions, counsel notes, and uploaded PDF exhibits.
   - Strictly isolated by User ID and Case ID; protected by IDOR authorization rules; never indexed into public knowledge.
4. **Tier 4: Synthetic Evaluation Benchmarks (Test Data):**
   - Fixed JSON test cases used by automated CI/CD suites (`backend/tests/evaluation/`).
   - Kept in version control; isolated from production database records.

Chapter 08 — API Keys, Environment Variables and External Integrations

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

> [!CAUTION] **Zero-Exposure Policy:** In accordance with strict security standards, no actual credentials, passwords, private keys, or raw JWT secrets appear in this document. All variables are documented by identifier, schema, purpose, and verified presence state.

1. Master Environment Variable Inventory

The following table documents every environment variable consumed by CivicSphere AI across backend services, frontend builds, database migrations, and container topologies.

Variable Name	Service / Provider	Purpose	Required?	Consuming File(s)	Present Locally?	Present in Production?	Sensitivity
ENVIRONMENT	Core Runtime	Defines environment mode (development, test, production)	No (Default: development)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Public
LOG_LEVEL	Core Logging	Controls log verbosity (DEBUG, INFO, WARNING, ERROR)	No (Default: INFO)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Public
APP_NAME	Core Runtime	Application identifier for telemetry and headers	No (Default: CivicSphere-AI)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Public
API_PREFIX	FastAPI Router	Base path prefix for all versioned REST endpoints	No (Default: /api/v1)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Public
ALLOWED_ORIGINS	FastAPI CORS	Whitelisted origins permitted to make cross-origin requests	Yes in Production	backend/app/common/config.py	**PRESENT**	**PRESENT**	Moderate
JWT_SECRET_KEY	Auth Subsystem	Secret key used to sign and verify HMAC-SHA256 JWT tokens	**REQUIRED (Fatal if missing)**	backend/app/common/config.py, auth/services.py	**PRESENT**	**PRESENT**	**CRITICAL**
JWT_ALGORITHM	Auth Subsystem	Cryptographic algorithm for JWT encoding (Default: HS256)	No	backend/app/common/config.py	**PRESENT**	**PRESENT**	Low
CSRF_SECRET_KEY	Auth Security	Entropy source for generating double-submit CSRF tokens	No (Has fallback)	backend/app/common/config.py	**PRESENT**	**PRESENT**	High
MAX_SOURCE_SIZE_BYTES	Ingestion Pipeline	Maximum allowed payload size for statutory documents (5 MB)	No	backend/app/common/config.py	**PRESENT**	**PRESENT**	Low
SSRF_PROTECTION_ENABLED	Ingestion Pipeline	Enforces DNS validation and blocks RFC1918 private IPs	No (Default: True)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Moderate
DATABASE_URL	PostgreSQL / SQLite	Connection URI for async SQLAlchemy relational database	**REQUIRED**	backend/app/common/database.py, alembic.ini	**PRESENT**	**PRESENT**	**CRITICAL**

Variable Name	Service / Provider	Purpose	Required?	Consuming File(s)	Present Locally?	Present in Production?	Sensitivity
DATABASE_POOL_SIZE	Database Engine	Maximum persistent connections in connection pool	No (Default: 10)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Low
NEO4J_URI	Neo4j Graph DB	Bolt connection URI for Neo4j instance	Optional (Has SQL fallback)	backend/app/common/config.py	**PRESENT**	**NOT REQUIRED**	Moderate
NEO4J_USER	Neo4j Graph DB	Username for Cypher graph queries (Default: neo4j)	Optional	backend/app/common/config.py	**PRESENT**	**NOT REQUIRED**	Low
NEO4J_PASSWORD	Neo4j Graph DB	Password for Neo4j authentication	**REQUIRED if Neo4j used**	backend/app/common/config.py	**PRESENT**	**NOT REQUIRED**	**CRITICAL**
REDIS_URL	Redis Cache	Redis connection string for distributed caching & rate limit	Optional (Has memory fallback)	backend/app/common/config.py	**PRESENT**	**NOT REQUIRED**	High
STORAGE_BACKEND	Document Store	Storage driver (local or supabase)	No (Default: local)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Moderate
STORAGE_LOCAL_PATH	Document Store	Local filesystem directory for uploaded documents	No (Default: ./uploads)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Low
SUPABASE_URL	Supabase Storage	REST URL of Supabase project instance	Conditional (If supabase)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Moderate
SUPABASE_SERVICE_ROLE_KEY	Supabase Storage	Privileged service key for bucket object uploads	Conditional (If supabase)	backend/app/common/config.py	**PRESENT**	**PRESENT**	**CRITICAL**
SUPABASE_STORAGE_BUCKET	Supabase Storage	Target bucket name (civicsphere-demo-documents)	No	backend/app/common/config.py	**PRESENT**	**PRESENT**	Low
EMBEDDING_PROVIDER	AI Engine	Active embedding provider (mock, openai, gemini, huggingface)	No (Default: mock)	backend/app/common/config.py	**PRESENT**	**PRESENT** (mock)	Low
EMBEDDING_MODEL	AI Engine	Target embedding model name	No (Default: text-embedding-3-small)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Low
EMBEDDING_DIMENSION	AI Engine	Target vector dimensionality (Enforced: 384)	No (Default: 384)	backend/app/common/config.py	**PRESENT**	**PRESENT**	Low
OPENAI_API_KEY	OpenAI API	Commercial API key for text-embedding-3-small and GPT-4	Optional	backend/app/common/providers/embedding.py	**ABSENT**	**ABSENT**	**CRITICAL**
GEMINI_API_KEY	Google Gemini API	Commercial API key for Gemini 1.5 Flash / Pro and embeddings	Optional	backend/app/common/providers/embedding.py	**ABSENT**	**ABSENT**	**CRITICAL**
HUGGINGFACE_API_KEY	Hugging Face API	API key for Hugging Face feature extraction endpoint	Optional	backend/app/common/providers/embedding.py	**ABSENT**	**ABSENT**	**CRITICAL**
ANTHROPIC_API_KEY	Anthropic API	API key for Claude 3.5 Sonnet LLM provider	Optional	backend/app/common/providers/llm.py	**ABSENT**	**ABSENT**	**CRITICAL**

Variable Name	Service / Provider	Purpose	Required?	Consuming File(s)	Present Locally?	Present in Production?	Sensitivity
NEXT_PUBLIC_API_URL	Frontend Web	Absolute URL of the backend API called by the browser	**REQUIRED for Web**	apps/web/.env.local, apps/web/src/lib/api-client.ts	**PRESENT**	**PRESENT**	Public

2. In-Depth Configuration Details

2.1 Critical Security Validations (`backend/app/common/config.py`)

CivicSphere AI includes startup sanity checks in `Settings.model_post_init()`:

1. **Placeholder Rejection:** The application refuses to start in `production` if `JWT_SECRET_KEY` or `NEO4J_PASSWORD` matches insecure placeholders (e.g. `change-this`, `CHANGE_ME`, `development-secret`).
2. **Minimum Entropy:** `JWT_SECRET_KEY` and `NEO4J_PASSWORD` must be at least 16 characters long.
3. **Wildcard CORS Prohibition:** In `production`, setting `ALLOWED_ORIGINS=["*"]` raises a fatal `ValueError`.

2.2 Frontend Public Variables vs. Private Secrets

- **Client-Exposed Variables:** Only variables prefixed with `NEXT_PUBLIC_` (specifically `NEXT_PUBLIC_API_URL`) are embedded in client-side JavaScript bundles.
- **Server-Side Protection:** `JWT_SECRET_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, and database passwords are never referenced in frontend code and remain strictly server-side.

3. Safe Configuration Checklist for New Developers

To configure an environment from scratch without risking secret exposure:

1. Copy `.env.example` to `.env`:
   ```
   cp .env.example .env
   ```
2. Generate a secure, 32-byte hexadecimal JWT secret key:
   ```
   python -c "import secrets; print(secrets.token_hex(32))"
   ```
   Paste the output as `JWT_SECRET_KEY` in `.env`.
3. Set `DATABASE_URL` to your local PostgreSQL instance:
   ```
   DATABASE_URL=postgresql+asyncpg://DB_USER:DB_PASSWORD@localhost:5432/DB_NAME
   ```
4. Keep `EMBEDDING_PROVIDER=mock` and `LLM_PROVIDER=mock` for zero-cost, offline local development.
5. In `apps/web/.env.local`, ensure:
   ```
   NEXT_PUBLIC_API_URL=http://localhost:8000
   ```

Chapter 09 — Authentication, Authorization and Security Architecture

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Authentication & Session Architecture

CivicSphere AI implements a defense-in-depth authentication framework built upon **JSON Web Tokens (JWT)** delivered via **secure, httpOnly cookies** combined with **Double-Submit CSRF Token protection**.

```
[Citizen Browser]                                          [FastAPI Backend]
       |                                                          |
       |--- 1. POST /api/v1/auth/login (email, password) --------->|
       |                                                          |-- Validate password (bcrypt, 12 rounds)
       |                                                          |-- If MFA enabled: request TOTP challenge
       |                                                          |-- Generate JWT (HS256, 24h expiry)
       |                                                          |-- Generate random CSRF token (32 bytes hex)
       |<-- 2. HTTP 200 OK ---------------------------------------|
       |       Set-Cookie: access_token=... (httpOnly, Secure)    |
       |       Set-Cookie: csrf_token=...   (Readable by JS)      |
       |                                                          |
       |--- 3. State-Changing Request (POST /api/v1/cases) ------->|
       |       Cookie: access_token=...                           |
       |       Header: X-CSRF-Token: <csrf_token_value>           |-- Verify access_token signature & expiry
       |                                                          |-- Verify X-CSRF-Token matches cookie
       |<-- 4. HTTP 200 / 201 Response --------------------------|
```

1.1 Password Hashing & Salt Security

- **Algorithm:** `bcrypt` with automatic salt generation using **12 work rounds**.
- **Implementation:** `backend/app/auth/models.py`:

```python
@staticmethod
def hash_password(password: str) -> str:
    salt = bcrypt.gensalt(rounds=12)
    return bcrypt.hashpw(password.encode("utf-8"), salt).decode("utf-8")
```

1.2 Cookie Security Configuration

To protect against Cross-Site Scripting (XSS) credential theft:

- The `access_token` cookie is marked **`httpOnly=True`**, rendering it inaccessible to client JavaScript.
- In production (`ENVIRONMENT="production"`), cookies are marked **`secure=True`** (HTTPS only) with **`samesite="none"`** to support cross-origin web portal requests from `civicsphere-web.vercel.app` to `civicsphere-backend.vercel.app`.
- In development, cookies use `samesite="lax"` and `secure=False` for local loopback testing.

1.3 CSRF Protection

To prevent Cross-Site Request Forgery:

- The backend issues a readable cookie named `csrf_token`.
- The frontend client (`apps/web/src/lib/api-client.ts`) extracts this value and transmits it in the `X-CSRF-Token` HTTP header on all `POST`, `PUT`, `DELETE`, and `PATCH` requests.

1.4 Multi-Factor Authentication (TOTP)

- **Standard:** RFC 6238 Time-based One-Time Password algorithm via `backend/app/auth/totp.py`.
- **Features:**
  - Setup endpoint (`POST /api/v1/auth/mfa/setup`) returns Base32 secret and `otpauth://` provisioning URI for Google Authenticator / Authy.
  - Verification endpoint (`POST /api/v1/auth/mfa/verify`) validates the initial 6-digit code before activating `mfa_enabled = True` in the database.
  - Disable endpoint (`POST /api/v1/auth/mfa/disable`) mandates both current account password and live TOTP token.

2. Role-Based Access Control (RBAC)

CivicSphere AI defines 5 canonical user roles via `backend/app/auth/models.py`:

Role Identifier	Description & System Permissions
CITIZEN	Default self-registered user. Can query legal guidance, check scheme eligibility, upload documents, and manage own cases.
RESEARCHER	Legal researcher or paralegal. Permitted to access unassigned open cases and provide counsel annotations.
REVIEWER	Senior legal reviewer. Permitted to review citizen guidance drafts and suggest edits to scheme procedural steps.
KNOWLEDGE_EDITOR	Statutory knowledge curator. Permitted to submit ingestion jobs and modify scheme eligibility rules.
ADMIN	System administrator. Permitted to view audit logs, security telemetry, trigger re-indexing, and manage accounts.

2.1 Privilege Escalation Guardrail

In `backend/app/auth/routes.py`:

```
requested_role = (payload.role or "CITIZEN").upper()
if settings.ENVIRONMENT == "production" and requested_role in ("ADMIN", "SYSTEM"):
    raise CivicSphereException(
        code="PRIVILEGE_ESCALATION_BLOCKED",
        message="Public self-registration for administrative roles is disabled.",
        status_code=403
    )
```

3. Data Isolation & IDOR Protection

CivicSphere AI enforces strict Insecure Direct Object Reference (IDOR) controls on case management and document retrieval:

- **Case Ownership:** In `backend/app/cases/routes.py::get_cases`, queries are filtered by role:
  - Citizens can *only* query records where `CaseModel.citizen_id == current_user.id`.
  - Researchers can query cases where `CaseModel.counsel_id == current_user.id` or `(counsel_id is None and status == "OPEN")`.
- **Single Record IDOR Validation:** In `backend/app/cases/routes.py::get_case_by_id`:
  - An explicit check verifies ownership before returning docket details. If a citizen attempts to access another user's `case_id`, the system raises HTTP 403 Forbidden.

4. Ingestion Security & SSRF Defense

When statutory knowledge is ingested via remote URL (`fetch_remote=True`), the pipeline enforces strict Server-Side Request Forgery (SSRF) checks (`backend/app/knowledge/ingestion/fetcher.py`):

1. **Protocol Whitelisting:** Restricts requests strictly to `http` and `https`. Schemes like `file://`, `ftp://`, or `gopher://` are rejected.
2. **DNS & Private IP Resolution:** Resolves target hostname prior to HTTP connection. If the target resolves to an RFC 1918 private IP (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `127.0.0.1/8`) or cloud metadata endpoints (`169.254.169.254`), the request is blocked.
3. **Audit Logging:** Every SSRF attempt triggers `AuditManager.record_security_event(threat_type="SSRF_INGESTION_ATTEMPT", severity="HIGH")`.

5. Document Upload & File Inspection Safeguards

In `backend/app/documents/routes.py` and `backend/app/document_processing/api/upload.py`:

- **Size Limit:** Maximum file size enforced at 10 MB.
- **Magic-Byte Signature:** File content must begin with `%PDF-` (`PDF_MAGIC_BYTES = b"%PDF-"`). Renamed executables (`.exe`, `.bat`, `.sh`) are rejected.
- **Malware & Active Script Scanning:** Uploaded PDF streams are scanned for embedded JavaScript (`/JavaScript`, `/JS`), launch actions (`/Launch`), or embedded executable streams before passing to PyMuPDF.

6. Rate Limiting Middleware

CivicSphere AI includes an in-memory token bucket rate limiter (`backend/app/common/rate_limiter.py`) registered in `main.py`:

- **Default Capacity:** 200 tokens per client IP.

- **Refill Rate:** 5.0 tokens per second.
- **Headers Returned:**
  - `X-RateLimit-Limit: 200`
  - `X-RateLimit-Remaining: <current_tokens>`
- **Exceeded Behavior:** Returns HTTP 429 Too Many Requests with `Retry-After` header.

Chapter 10 — System Architecture and End-to-End Data Flows

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. System Context Diagram (C4 Level 1)

```
flowchart TD
    Citizen["Indian Citizen / Public User"]
    Researcher["Legal Researcher / Paralegal"]
    Admin["System Administrator"]

    subgraph CivicSpherePlatform["CivicSphere AI Platform"]
        WebPortal["Next.js 15 Web Portal\n(App Router / Turbopack)"]
        BackendAPI["FastAPI Modular Monolith\n(Unified Backend / Python 3.12)"]
        PostgreSQL["Durable Database & Vector Index\n(PostgreSQL 17.6 + pgvector)"]
        Neo4j["Knowledge Graph\n(Neo4j 5.15 / Relational Graph)"]
    end

    ExternalIndiaCode["Official India Code\n(Legislative Dept, Law Ministry)"]
    ExternalMeitY["MeitY & Gov Portals\n(Gazette & Scheme Portals)"]

    Citizen -->|"Asks civic/legal queries,\nchecks schemes, uploads docs"| WebPortal
    Researcher -->|"Reviews open cases,\ninspects statutory graph"| WebPortal
    Admin -->|"Monitors audit telemetry,\ntriggers reindexing"| WebPortal

    WebPortal -->|"HTTPS REST API (/api/v1/*)\nwith Cookies + CSRF Header"| BackendAPI

    BackendAPI -->|"ACID Relational Writes\n& pgvector Similarity"| PostgreSQL
    BackendAPI -->|"Entity Traversal &\nNeighborhood Queries"| Neo4j
    BackendAPI -.->|"Fetches verified text\n(with SSRF checks)"| ExternalIndiaCode
    BackendAPI -.->|"Fetches gazette updates"| ExternalMeitY
```

Narrative 1: System Context

The CivicSphere AI platform acts as a secure intermediary between ordinary citizens, legal professionals, and authoritative Indian statutory repositories. Users access the platform via a responsive web client hosted on Vercel. All state-changing and analytical requests flow to the FastAPI backend, which coordinates relational persistence in PostgreSQL, vector searches in pgvector, and graph traversals.

2. Frontend / Backend / Database Architecture (C4 Level 2)

```
flowchart LR
    subgraph ClientLayer["Client Layer (Browser)"]
        UI["React 19 Components\n(Assistant, Schemes, Cases)"]
        ApiClient["Universal ApiClient\n(api-client.ts)"]
    end

    subgraph EdgeLayer["Edge / Routing Layer (Vercel)"]
        NextServer["Next.js Server / Proxy\n(civicsphere-web.vercel.app)"]
    end

    subgraph BackendLayer["Backend Application Layer (FastAPI)"]
        MainApp["FastAPI Main App & Lifespan\n(backend/app/main.py)"]
        RateLimiter["TokenBucket RateLimiter"]
        SecurityMW["Security & Correlation MW"]

        subgraph Subsystems["Core Subsystems"]
            AuthMod["Auth Subsystem\n(JWT, RBAC, TOTP)"]
            LegalMod["Legal Guidance Engine\n(Module A)"]
            GovMod["Government Scheme Nav\n(Module B)"]
            DocMod["Document Processing\n(Module D, OCR)"]
            CaseMod["Case Workspace\n(Module Cases)"]
            KnowMod["Knowledge & Hybrid RAG\n(Module C)"]
            AgentMod["Multi-Agent System\n(Phase 6 Orchestrator)"]
        end
    end

    subgraph DataLayer["Persistence & Storage Layer"]
        PG["PostgreSQL 17.6 (Supabase)\n- users, cases, documents\n- knowledge_chunks\n- knowledge_embeddings (pgvector)\n- knowledge_graph_nodes\n- knowledge_graph_edges"]
        LocalDisk["Local File Storage / MinIO\n(uploads/*.pdf)"]
    end

    UI --> ApiClient
    ApiClient -->|"HTTPS JSON / FormData"| NextServer
    NextServer -->|"Proxy Rewrites"| MainApp
    MainApp --> RateLimiter --> SecurityMW
    SecurityMW --> Subsystems
    Subsystems -->|"Async Sessions (SQLAlchemy)"| PG
    DocMod -->|"PDF Binary Writes"| LocalDisk
```

Narrative 2: Application Architecture

The architecture is designed as a **Modular Monolith**:

- Code is partitioned into strictly bounded domains (`auth`, `legal`, `government`, `knowledge`, `documents`, `cases`, `agents`, `audit`).
- Modules share a single async database connection pool, eliminating distributed transaction overhead while preserving domain boundaries.
- Cross-module communication is performed via clean Python service calls (e.g. `LegalGuidanceEngine` directly invokes `hybrid_retrieval_service.retrieve`).

3. End-to-End Legal Question-to-Answer Sequence

```
sequenceDiagram
    autonumber
    actor Citizen as Citizen / User
    participant Web as Web Portal (/legal)
    participant API as FastAPI Router
    participant Engine as LegalGuidanceEngine
    participant Guard as CircuitBreaker / Security
    participant RAG as HybridRetrievalService
    participant PG as PostgreSQL (pgvector)
    participant Graph as GraphRepository

    Citizen->>Web: Enters query ("What is RTI appeal timeline?")
    Web->>API: POST /api/v1/legal/query { query, jurisdiction: "IN" }
    API->>Engine: process_legal_query(query)
    Engine->>Guard: evaluate_query_relevance(query)
    Guard-->>Engine: Status: ALLOWED (Relevance = High)

    Engine->>RAG: retrieve(RetrievalRequest)
    par Lexical Matching
        RAG->>RAG: Compute BM25 token scores on statutory sections
    and Vector Similarity
        RAG->>PG: Cosine similarity search over knowledge_chunks
        PG-->>RAG: Top vector candidate chunks
    and Graph Expansion
        RAG->>Graph: Traverse 1-hop neighborhood of extracted legal entities
        Graph-->>RAG: Connected statutory nodes (CIC, First Appellate)
    end

    RAG->>RAG: Combine candidates via Reciprocal Rank Fusion (k=60)
    RAG->>RAG: Assemble EvidencePack (Chunks, Sections, Citations)
    RAG-->>Engine: EvidencePack (Confidence: 0.98)

    Engine->>Engine: Synthesize 10-section structured legal guidance
    Engine->>Engine: Attach official India Code citations
    Engine-->>API: LegalAnswer (Confidence: 0.98, Citations: RTI Act Sec 19)
    API-->>Web: HTTP 200 ApiResponse[LegalAnswer]
    Web-->>Citizen: Renders 10 accordion cards & statutory badges
```

Narrative 3: Legal Query Flow

1. Citizen inputs query in web interface.
2. The query is posted to `/api/v1/legal/query`.
3. The circuit breaker verifies legal and civic relevance.
4. The hybrid retrieval engine concurrently queries BM25 lexical index, pgvector embeddings, and knowledge graph neighborhood.
5. Reciprocal Rank Fusion merges rankings to select top 5 authoritative chunks.
6. A 10-section structured answer is assembled, citing specific sections and India Code URLs.
7. Web portal renders results with an overall confidence score (0.98).

4. Ingestion Pipeline Sequence Flow

```
sequenceDiagram
    autonumber
    actor Admin as Admin / Seed Runner
    participant Ingest as IngestionPipeline
    participant Fetcher as SourceFetcher
    participant Cleaner as ContentCleaner & Chunker
    participant Extractor as Entity & Relationship Extractor
    participant Embedder as ChunkEmbedder
    participant DB as PostgreSQL 17.6

    Admin->>Ingest: ingest_source(title, publisher, official_url, raw_text)
    Ingest->>Ingest: Check SSRF safety (blocks RFC1918 private IPs)
    Ingest->>DB: Register source in knowledge_sources (Status: PROCESSING)
    Ingest->>Cleaner: Clean text and split on section boundaries
    Cleaner-->>Ingest: Chunks (300-500 words per chunk)

    Ingest->>Extractor: Extract legal entities and relationships
    Extractor-->>Ingest: Entities (Acts, Sections, Bodies) & Edges (ESTABLISHES, APPEALS_TO)
```

```
    Ingest->>Embedder: embed_chunks(chunks)
    Embedder-->>Ingest: 384-dimensional normalized vectors

    par Persist Chunks & Vectors
        Ingest->>DB: INSERT into knowledge_chunks & knowledge_embeddings
    and Persist Graph
        Ingest->>DB: INSERT into knowledge_graph_nodes & knowledge_graph_edges
    end

    Ingest->>DB: UPDATE knowledge_sources (Status: COMPLETED, Version: 1)
    Ingest->>DB: INSERT into audit_events (Event: KNOWLEDGE_INGESTION_COMPLETED)
    Ingest-->>Admin: Ingestion summary (Chunks: 4, Entities: 9, Relationships: 8)
```

Narrative 4: Ingestion Pipeline

When statutory acts are ingested:

1. Source URL is evaluated for trust level and SSRF safety.
2. The source record is created in `knowledge_sources` with a SHA-256 content hash.
3. The text is parsed into structured section chunks.
4. Entity and relationship extractors build graph nodes and edges.
5. The chunk embedder generates 384-dimensional dense vectors.
6. Chunks, vectors, graph nodes, and edges are committed in a single atomic database transaction.
7. An audit event is recorded for compliance.

5. Multi-Agent Orchestration Flow

```
flowchart TD
    UserQuery["User Query: Citizen Issue"] --> Orchestrator["Agent Orchestrator\n(orchestrator.py)"]

    Orchestrator --> SafetyCheck["SafetyAgent: Input Sanitization\n& PII Masking"]
    SafetyCheck --> Planner["Generate Bounded 5-Step Execution Plan"]

    Planner --> Step1["Step 1: Consult KnowledgeAgent\n(Execute Hybrid Retrieval)"]
    Step1 --> Step2["Step 2: Consult LegalAgent\n(Check Limitation Deadlines)"]
    Step2 --> Step3["Step 3: Consult CitationAgent\n(Verify Section Validity)"]

    Step3 --> Synthesizer["Synthesize Unified Agent Response"]
    Synthesizer --> AuditLog["Log Execution Trace & Tools Used"]
    AuditLog --> FinalResponse["Return ApiResponse to Client"]
```

Narrative 5: Multi-Agent Coordination

The multi-agent orchestrator decomposes complex citizen requests into a bounded sequence of specialist agent tasks:

- `SafetyAgent` pre-screens inputs for prompt injection and PII.
- `KnowledgeAgent` retrieves grounded statutory evidence.
- `LegalAgent` analyzes procedural remedies and limitation periods.
- `CitationAgent` confirms that all cited enactments exist in the authoritative registry.
- Execution is strictly bounded to a maximum of 5 steps to prevent infinite execution loops.

Chapter 11 — Installation, Configuration and Build from Scratch

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. System Prerequisites

Before setting up CivicSphere AI, ensure the following developer tools are installed on your host machine:

Tool / Runtime	Minimum Version	Recommended Version	Purpose
Operating System	Windows 10/11, Ubuntu 20.04+, macOS 12+	Windows 11 / Ubuntu 22.04 LTS	Host OS
Git	2.34.0	2.45+	Version control
Python	3.10.0	3.12.x	Backend runtime
Node.js	18.18.0	20.x LTS or 22.x	Frontend runtime
npm	9.0.0	10.x	Node package manager
Docker Desktop	24.0.0	Latest (Compose v2)	Local database & infrastructure stack

2. Step-by-Step Clean Checkout Setup

Step 1: Clone Repository

```
git clone https://github.com/Khushbu112005/Finalyear_MajorProject.git
cd Finalyear_MajorProject
```

Step 2: Start Infrastructure Stack (Docker)

The local development environment uses Docker Compose to provision PostgreSQL with pgvector, Neo4j, Redis, and MinIO:

```
docker compose -f infrastructure/docker/docker-compose.yml up -d
```

Verify all 4 core containers are running:

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Expected running containers:

- `civicsphere-postgres` (`0.0.0.0:5432->5432/tcp`)
- `civicsphere-neo4j` (`0.0.0.0:7474->7474/tcp`, `0.0.0.0:7687->7687/tcp`)
- `civicsphere-redis` (`0.0.0.0:6379->6379/tcp`)
- `civicsphere-minio` (`0.0.0.0:9000->9000/tcp`, `0.0.0.0:9001->9001/tcp`)

Step 3: Backend Python Environment & Dependencies

Create and activate an isolated Python 3.12 virtual environment:

On Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

On Linux / macOS (Bash):

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Step 4: Configure Backend Environment Variables

Copy the provided `.env.example` file to create your local `.env`:

```
cp .env.example .env
```

Generate a secure 32-byte hexadecimal secret key:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Edit `.env` to set:

```
ENVIRONMENT=development
LOG_LEVEL=INFO
APP_NAME=CivicSphere-AI
API_PREFIX=/api/v1
ALLOWED_ORIGINS=["http://localhost:3000","http://127.0.0.1:3000"]
JWT_SECRET_KEY=<paste_generated_32_byte_key_here>
JWT_ALGORITHM=HS256

DATABASE_URL=postgresql+asyncpg://DB_USER:DB_PASSWORD@localhost:5432/DB_NAME
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=REPLACE_WITH_A_UNIQUE_LOCAL_SECRET

EMBEDDING_PROVIDER=mock
EMBEDDING_MODEL=text-embedding-3-small
EMBEDDING_DIMENSION=384
LLM_PROVIDER=mock
RERANKER_PROVIDER=heuristic
```

Step 5: Execute Database Migrations

Apply the version-controlled Alembic schema migrations:

```
alembic upgrade head
```

Expected output:

```
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> 0001, initial canonical schema
INFO  [alembic.runtime.migration] Running upgrade 0001 -> 0002, sync canonical models
INFO  [alembic.runtime.migration] Running upgrade 0002 -> 0003, pgvector and durable knowledge
```

Step 6: Seed Verified Statutory Knowledge Base

Populate the knowledge repository with the Right to Information Act, DPDP Act, and Consumer Protection Act:

```
python -c "import asyncio; from data.seed.seed_data import seed_knowledge_base; asyncio.run(seed_knowledge_base())"
```

Step 7: Frontend Dependencies & Configuration

Install frontend workspace dependencies:

```
cd apps/web
npm install
```

Create or verify `apps/web/.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

Return to repository root:

```
cd ../..
```

3. Starting the Development Servers

Terminal 1: Backend FastAPI Server

From the repository root (with `.venv` activated):

```
uvicorn backend.app.main:app --host 0.0.0.0 --port 8000 --reload
```

- Liveness check: `curl http://localhost:8000/api/v1/health`
- Readiness check: `curl http://localhost:8000/api/v1/health/ready`
- Interactive OpenAPI Docs: `http://localhost:8000/docs`

Terminal 2: Frontend Next.js Server

From `apps/web`:

```
cd apps/web
npm run dev
```

Open your browser at: `http://localhost:3000`

4. Verification & Testing Commands

To verify full system integrity from the repository root:

```
# 1. Run full backend automated test suite (91 passing tests)
python -m pytest -p no:pytest_ethereum -q

# 2. Run frontend lint check
npm run lint --workspace=civicsphere-web

# 3. Run frontend production compilation build
npm run build --workspace=civicsphere-web
```

5. Troubleshooting Common Setup Issues

Symptom / Error	Root Cause	Solution
FATAL: JWT_SECRET_KEY is missing or too short (min 16 chars)	.env has missing or default placeholder key.	Generate a 32-byte key via secrets.token_hex(32) and set in .env.
psycopg2.OperationalError / ConnectionRefusedError: port 5432	Docker PostgreSQL container is not started.	Run docker compose -f infrastructure/docker/docker-compose.yml up -d.
prepared statement "..." already exists	Using Supabase transaction pooler without statement cache disable.	Ensure connect_args["statement_cache_size"] = 0 in database.py (already configured).
ModuleNotFoundError: No module named 'packages'	Python path does not include repository root.	Run Python commands from repository root with .venv active.
Next.js API requests fail with CORS Error	Backend ALLOWED_ORIGINS does not include http://localhost:3000.	Add http://localhost:3000 to ALLOWED_ORIGINS in .env.

Chapter 12 — Testing, QA, Benchmarks and Verification Evidence

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026
Latest Automated Test Run: 91 Passed, 1 Skipped, 0 Failed (Execution time: 22.43s)

1. Test Suite Inventory & Structure

CivicSphere AI maintains a rigorous test pyramid organized under `backend/tests/` and `packages/schemas/`, spanning isolated unit tests, API contract tests, security suites, and end-to-end integration journeys.

Suite Category	Directory / File	Tests	Primary Objective & Coverage	Mock Dependencies	Latest Result
API Endpoints	tests/api/test_api_endpoints.py	12	Health probes, router prefixing, global error envelopes	Async test client	**PASSED**
Authentication API	tests/api/test_auth_api.py	8	User registration, login, cookie setting, CSRF header enforcement	Mock DB session	**PASSED**
Legal Guidance API	tests/api/test_legal_api.py	6	Statutory query submission, 10-section response structure	In-memory RAG	**PASSED**
Government API	tests/api/test_government_api.py	6	Scheme eligibility rules, demographic parameter parsing	Mock DB session	**PASSED**
Cases Workspace API	tests/api/test_cases_api.py	7	Case CRUD lifecycle, status updates, IDOR checks	Mock DB session	**PASSED**
Document Pipeline API	tests/api/test_document_pipeline_api.py	5	Multipart PDF upload, magic-byte checks, OCR pipeline	Synthetic PDF bytes	**PASSED**
Agents API	tests/api/test_agents_api.py	4	Multi-agent chat invocation, tool registry catalog	Sandboxed tools	**PASSED**
Cross-Module E2E	tests/integration/test_flagship_cross_module_e2e.py	4	Complete journey: Register -> Upload Doc -> File Case -> Query Legal	Full app integration	**PASSED**
Hybrid Retrieval	tests/integration/test_hybrid_retrieval.py	5	BM25 lexical + vector cosine + graph traversal fusion	In-memory graph/vector	**PASSED**
Ingestion Pipeline	tests/integration/test_ingestion_pipeline.py	4	13-stage ingestion sequence from raw text to indexed state	Mock embedder	**PASSED**
Knowledge Agent	tests/integration/test_knowledge_agent.py	3	Agentic tool invocation for statutory section lookup	In-memory registry	**PASSED**
Live Smoke Tests	tests/integration/test_live_sources_smoke.py	2	Smoke verification of live gazette URLs	External HTTP (skipped offline)	**1 Passed, 1 Skipped**
IDOR & Auth Security	tests/security/test_idor_and_auth.py	4	Cross-user case tampering attempts, admin role escalation	Multi-user fixtures	**PASSED**
SSRF Defense	tests/security/test_ssrf_defense.py	4	RFC 1918 private IP blocking during remote source ingestion	Fake DNS resolver	**PASSED**
Prompt Injection	tests/security/test_prompt_injection.py	3	Jailbreak heuristics and system instruction override attempts	Attack vector payloads	**PASSED**
Poisoning Defense	tests/security/test_poisoning_defense.py	2	Tampered chunk hash detection and rejection	Injected invalid hash	**PASSED**
Cypher Injection	tests/security/test_cypher_injection.py	2	Parameterized Cypher queries preventing malicious graph syntax	Unsanitized input strings	**PASSED**

Suite Category	Directory / File	Tests	Primary Objective & Coverage	Mock Dependencies	Latest Result
Rate Limiter	tests/security/ test_rate_limiter.py	3	Token bucket exhaustion and HTTP 429 Retry-After verification	Fast loop requests	**PASSED**
PII Sanitization	tests/security/ test_pii_sanitization.py	2	Aadhaar, PAN card, and phone number regex redaction	Regex test fixtures	**PASSED**
Chunker Unit	tests/unit/ test_chunker.py	3	Section boundary splitting, word count limits	Statutory strings	**PASSED**
Source Registry Unit	tests/unit/ test_source_registry.py	4	Idempotent registration, version bumping, status tracking	In-memory registry	**PASSED**
Extractors Unit	tests/unit/ test_extractors.py	3	Legal entity and statutory relationship extraction	Sample legal text	**PASSED**
Reranker Unit	tests/unit/ test_reranker.py	2	Reciprocal Rank Fusion calculation accuracy	Ranked lists	**PASSED**
Database Migrations	tests/unit/ test_alembic.py	1	Alembic migration heads and configuration check	Alembic config	**PASSED**
Pydantic Contracts	packages/schemas/ test_contracts.py	4	ApiResponse envelope serialization and validation	Pydantic models	**PASSED**

2. Latest Automated Test Execution Evidence

The test suite is executed using `pytest` from the repository root:

```
python -m pytest -p no:pytest_ethereum -q
```

Sanitized Execution Output

```
.....................................................................................s......... [100%]
============================== 91 passed, 1 skipped in 22.43s ==============================
```

- **Total Tests Discovered:** 92
- **Passed:** 91 (98.9%)
- **Skipped:** 1 (Live external network smoke test skipped when running in offline CI mode)
- **Failed:** 0 (0.0%)

3. Real PostgreSQL 10-Step User Journey Verification

To independently verify genuine end-to-end database persistence on PostgreSQL (Supabase / Docker port 5432) rather than ephemeral SQLite or memory fallbacks, a dedicated integration script (`scratch/test_real_postgres_journey.py`) was executed:

Step #	User Journey Action	Target Endpoint / Function	Verification Evidence	Result
Step 1	User Registration	POST /api/v1/auth/register	Synthetic citizen account registered; HTTP 200 returned; httpOnly cookies set	**PASSED**
Step 2	DB Record Inspection	SQL query on users table	Directly queried PostgreSQL users table; row verified with bcrypt hash	**PASSED**
Step 3	User Authentication	POST /api/v1/auth/login	Authenticated with registered credentials; JWT issued and validated	**PASSED**
Step 4	Knowledge Ingestion	seed_knowledge_base()	Ingested RTI Act, DPDP Act, Consumer Protection Act into PostgreSQL	**PASSED**
Step 5	Grounded Legal Query	POST /api/v1/legal/query	Queried Constitution Article 21; returned confidence 0.98 with genuine statutory citations	**PASSED**
Step 6	Graph Traversal	GET /api/v1/knowledge/entities/...	Traversed 1-hop neighborhood; verified 34 nodes and active statutory relationships	**PASSED**

Step #	User Journey Action	Target Endpoint / Function	Verification Evidence	Result
Step 7	Circuit Breaker Test	POST /api/v1/legal/query	Submitted out-of-domain query (“Quantum gravity in black holes”); rejected with confidence 0.0	**PASSED**
Step 8	Relational Edge Verify	SQL query on knowledge_graph_edges	Directly queried PostgreSQL; verified directed edges (ESTABLISHES, APPEALS_TO)	**PASSED**
Step 9	Cache Eviction & Hydration	Memory wipe + hydrate_from_db()	Cleared all in-memory registries; triggered hydration; restored 3 sources, 96 nodes, 12 vectors	**PASSED**
Step 10	Synthetic Account Cleanup	SQL DELETE FROM users WHERE id=...	Removed test user from database; verified audit event logged	**PASSED**

4. Retrieval Benchmarks & Evaluation Metrics

The retrieval subsystem was benchmarked against the annotated test queries in `data/evaluation/dataset.json` using the evaluation runner in `backend/app/knowledge/evaluation/retrieval_metrics.py`:

Metric	Target Threshold	Measured Score (Local Deterministic)	Description
Precision@5	≥ 0.80	**0.88**	Proportion of top-5 retrieved chunks that directly cite the applicable statutory section
Recall@5	≥ 0.75	**0.84**	Proportion of all relevant ground-truth sections successfully present in top-5 chunks
Mean Reciprocal Rank (MRR)	≥ 0.85	**0.91**	Average reciprocal rank of the first relevant statutory chunk across all benchmark queries
Out-of-Domain Rejection Rate	100%	**100% (40/40)**	Successful rejection of non-civic/non-legal queries with confidence 0.0
Prompt Injection Interception	$\geq 95\%$	**100% (25/25)**	Interception of jailbreak tokens, system override prompts, and roleplay commands

Chapter 13 — Deployment, CI/CD, Infrastructure and Operations

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Dual Vercel Production Cloud Architecture

CivicSphere AI is hosted in production across two decoupled, synchronized Vercel cloud projects:

```
[Citizen Client]
       |
       +---> 1. Frontend Web Portal: https://civicsphere-web.vercel.app
       |      - Framework: Next.js 15 App Router (Node.js 22.x)
       |      - Deployed via: npx vercel --prod in apps/web
       |      - Active Deployment ID: dpl_5E9C9wjft69ERYSm2xWcociPe6zL
       |
       +---> 2. Unified Backend API: https://civicsphere-backend.vercel.app
              - Framework: FastAPI / ASGI (Python 3.12 Serverless)
              - Entrypoint: api/index.py -> backend.app.main:app
              - Active Deployment ID: dpl_6d5PbXUoHi4ufTe1PP965LDjttgd
              - Persistence: Supabase PostgreSQL 17.6 + pgvector (Port 6543)
```

1.1 Backend Vercel Configuration (`vercel.json`)

```json
{
  "version": 2,
  "builds": [
    {
      "src": "api/index.py",
      "use": "@vercel/python"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "api/index.py"
    }
  ]
}
```

Note on Vercel Python Compatibility: Vercel v2 configurations using `@vercel/python` strictly prohibit combining `"functions"` with `"builds"`. Commit `b566c93` removed the conflicting `"functions"` key, ensuring instant, error-free automated builds.

1.2 Frontend Vercel Configuration (`apps/web/.vercel/project.json`)

```json
{
  "projectId": "prj_e8qZuiSm9DKFb5ZL3GgCO2UjsKGT",
  "orgId": "team_FWMl2iG5JaXnfIBbw6Samefu",
  "projectName": "civicsphere-web",
  "settings": {
    "framework": "nextjs",
    "nodeVersion": "24.x"
  }
}
```

- **Environment Variable:** `NEXT_PUBLIC_API_URL="https://civicsphere-backend.vercel.app"` is configured in Vercel project production settings.

2. Docker Compose 7-Service Topology

For local development, automated testing, and offline deployment demonstrations, the complete platform runs via `infrastructure/docker/docker-compose.yml`:

```yaml
version: '3.8'

services:
  # 1. Primary Relational DB with Vector Extension
  postgres:
    image: pgvector/pgvector:pg16
    container_name: civicsphere-postgres
    restart: unless-stopped
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U civicsphere_user -d civicsphere"]

  # 2. Graph Database
  neo4j:
    image: neo4j:5.15.0
    container_name: civicsphere-neo4j
```

```
    restart: unless-stopped
    ports: ["7474:7474", "7687:7687"]
    volumes: [neo4jdata:/data]

  # 3. Cache & Rate Limiting
  redis:
    image: redis:7.2-alpine
    container_name: civicsphere-redis
    ports: ["6379:6379"]

  # 4. S3-Compatible Object Store
  minio:
    image: minio/minio:latest
    container_name: civicsphere-minio
    ports: ["9000:9000", "9001:9001"]
    volumes: [miniodata:/data]

  # 5. Modular Monolith Backend
  backend:
    build:
      context: ../..
      dockerfile: infrastructure/docker/Dockerfile.backend
    container_name: civicsphere-backend
    ports: ["8000:8000"]
    depends_on:
      postgres: { condition: service_healthy }

  # 6. Next.js Frontend Web Client
  frontend:
    build:
      context: ../../apps/web
      dockerfile: Dockerfile
    container_name: civicsphere-frontend
    ports: ["3000:3000"]
    depends_on: [backend]

  # 7. Continuous Source Synchronization Scheduler Daemon
  sync-cron:
    build:
      context: ../..
      dockerfile: infrastructure/docker/Dockerfile.backend
    container_name: civicsphere-sync-cron
    command: ["python", "infrastructure/scripts/sync_sources_cron.py"]
```

3. GitHub Actions CI/CD Pipeline

The project includes two GitHub Actions workflows under `.github/workflows/`:

3.1 Continuous Integration (`.github/workflows/ci.yml`)

- **Triggers:** Push to `main` branch or pull requests targeting `main`.
- **Jobs:**
  1. **Backend Tests:** Provisions Python 3.12, installs `requirements.txt`, executes `pytest` suite across all 91 tests.
  2. **Frontend Lint & Build:** Provisions Node.js 20, installs npm dependencies, runs `npm run lint` and `npm run build`.

3.2 Security Scanning (`.github/workflows/security.yml`)

- **Triggers:** Weekly scheduled cron and manual workflow dispatch.
- **Tools:**
  - `safety`: Scans Python dependencies for known CVEs.
  - `bandit`: Static analysis for Python security vulnerabilities.
  - `npm audit`: Scans JavaScript packages for critical advisories.

4. Operations & Runbooks

4.1 Deployment Runbook

To deploy code changes to production:

```
# 1. Commit and push changes to GitHub main branch
git add .
git commit -m "feat/fix: descriptive commit message"
git push origin main

# 2. Deploy Backend to Vercel
npx vercel --prod --yes

# 3. Deploy Frontend to Vercel
cd apps/web
```

```
npx vercel --prod --yes
cd ../..

# 4. Verify live production endpoints
curl https://civicsphere-backend.vercel.app/api/v1/health
curl https://civicsphere-backend.vercel.app/api/v1/health/ready
curl -I https://civicsphere-web.vercel.app
```

4.2 Rollback Procedure

If a production deployment introduces regressions:

1. Identify the previous stable Vercel deployment ID via:
   ```
   npx vercel list civicsphere-backend
   ```
2. Promote the previous stable deployment:
   ```
   npx vercel rollback <deployment-id> --yes
   ```
3. Revert the corresponding Git commit:
   ```
   git revert HEAD
   git push origin main
   ```

4.3 Log Inspection & Diagnostics

To inspect live production serverless logs in real time:

```
npx vercel logs civicsphere-backend.vercel.app
```

To inspect live frontend web logs:

```
npx vercel logs civicsphere-web.vercel.app
```

Chapter 14 — Complete Feature Traceability Matrix

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Feature Traceability Matrix

The following matrix establishes strict bidirectional traceability from the approved PRD and 97-Page Master Specification requirements to the implementing frontend components, API endpoints, backend services, database tables, and automated test assertions.

Requirement ID	Requirement Title & Description	Frontend Route & Component	API Endpoint	Implementing Backend Service	Database Model / Provider	Verification Test File	Status
REQ-AUTH-01	**Citizen Self-Registration**Allows citizens to create accounts with email, password, and profile metadata.	/ registerapps/web/src/app/register/page.tsx	POST /api/v1/auth/register	backend/app/auth/services.py::register_user	UserModel (users table)	tests/api/test_auth_api.py	**RUNTIME-VERIFIED**
REQ-AUTH-02	**Session Authentication**Issues httpOnly JWT cookie and double-submit CSRF token on login.	/loginapps/web/src/app/login/page.tsx	POST /api/v1/auth/login	backend/app/auth/services.py::authenticate_user	UserModel	tests/api/test_auth_api.py	**RUNTIME-VERIFIED**
REQ-AUTH-03	**TOTP Multi-Factor Auth**RFC 6238 compliant two-factor authentication challenge and verification.	/login (Modal)apps/web/src/app/login/page.tsx	POST /api/v1/auth/mfa/setupPOST /api/v1/auth/mfa/verify	backend/app/auth/totp.py::TOTPManager	UserModel.mfa_secret	tests/api/test_auth_api.py	**SOURCE-VERIFIED**
REQ-AUTH-04	**Privilege Escalation Prevention**Blocks self-registration for ADMIN and SYSTEM roles in production.	/ registerapps/web/src/app/register/page.tsx	POST /api/v1/auth/register	backend/app/auth/routes.py::register	UserModel.role	tests/security/test_idor_and_auth.py	**TEST-VERIFIED**
REQ-LEG-01	**Structured Legal Guidance**Generates 10-section structured legal guidance grounded on statutory acts.	/legalapps/web/src/app/legal/page.tsx	POST /api/v1/legal/query	backend/app/legal/services/legal_engine.py	KnowledgeSourceModelKnowledgeChunkModel	tests/api/test_legal_api.py	**RUNTIME-VERIFIED**
REQ-LEG-02	**Out-of-Domain Circuit Breaker**Rejects non-civic/non-legal queries with confidence 0.0 and clear disclaimers.	/legalapps/web/src/app/legal/page.tsx	POST /api/v1/legal/query	backend/app/legal/services/legal_engine.py	Heuristic query classifier	tests/api/test_legal_api.py	**RUNTIME-VERIFIED**
REQ-LEG-03	**Statutory Acts Registry**Returns verified central/state legislation indexed in the platform.	/ sourcesapps/web/src/app/sources/page.tsx	GET /api/v1/legal/acts	backend/app/knowledge/sources/registry.py	KnowledgeSourceModel	tests/api/test_legal_api.py	**RUNTIME-VERIFIED**

Requirement ID	Requirement Title & Description	Frontend Route & Component	API Endpoint	Implementing Backend Service	Database Model / Provider	Verification Test File	Status
REQ-GOV-01	**Scheme Problem Analysis**Extracts civic domain and demographic variables from natural language queries.	/ governmentapps/web/src/app/government/page.tsx	POST /api/v1/government/analyze	backend/app/government/services/ai_service.py	GovernmentServiceModel	tests/api/test_government_api.py	**TEST-VERIFIED**
REQ-GOV-02	**Scheme Eligibility Evaluation**Evaluates age, income, and category criteria against welfare rules.	/ governmentapps/web/src/app/government/page.tsx	POST /api/v1/government/check-eligibility	backend/app/government/services/eligibility.py	GovernmentServiceModel	tests/api/test_government_api.py	**TEST-VERIFIED**
REQ-DOC-01	**Secure PDF Document Upload**Validates %PDF-magic-byte signature and limits uploads to 10 MB.	/ documentsapps/web/src/app/documents/page.tsx	POST /api/v1/documents/upload	backend/app/documents/services/pipeline.py	DocumentModel (documents table)	tests/api/test_document_pipeline_api.py	**TEST-VERIFIED**
REQ-DOC-02	**OCR Text Extraction**Extracts structural plain text and metadata from PDF files using PyMuPDF.	/ documentsapps/web/src/app/documents/page.tsx	POST /api/v1/documents/upload	backend/app/document_processing/services/ocr_service.py	PyMuPDF library (fitz)	tests/api/test_document_pipeline_api.py	**TEST-VERIFIED**
REQ-DOC-03	**Knowledge Graph Linkage**Links uploaded citizen documents to related statutory acts and open cases.	/ documentsapps/web/src/app/documents/page.tsx	POST /api/v1/documents/upload	backend/app/documents/services/knowledge_linker.py	KnowledgeGraphEdgeModel	tests/api/test_document_pipeline_api.py	**TEST-VERIFIED**
REQ-CASE-01	**Citizen Case Workspace**Provides case lifecycle CRUD, filing deadlines, and procedural milestones.	/casesapps/web/src/app/cases/page.tsx	GET /api/v1/casesPOST /api/v1/cases	backend/app/cases/routes.py	CaseModel (cases table)	tests/api/test_cases_api.py	**TEST-VERIFIED**
REQ-CASE-02	**IDOR Access Control**Restricts case visibility to case owners, assigned counsel, and system admins.	/casesapps/web/src/app/cases/page.tsx	GET /api/v1/cases/{case_id}	backend/app/cases/routes.py::get_case_by_id	CaseModel	tests/security/test_idor_and_auth.py	**TEST-VERIFIED**
REQ-KNOW-01	**13-Stage Ingestion Pipeline**End-to-end statutory document ingestion, chunking, and dual indexing.	/admin/sourcesapps/web/src/app/admin/sources/page.tsx	POST /api/v1/knowledge/ingestion/jobs	backend/app/knowledge/ingestion/pipeline.py	KnowledgeSourceModelKnowledgeChunkModel	tests/integration/test_ingestion_pipeline.py	**RUNTIME-VERIFIED**
REQ-KNOW-02	**pgvector Embedding Storage**Durable vector persistence in PostgreSQL with cosine similarity search.	/ knowledgeapps/web/src/app/knowledge/page.tsx	POST /api/v1/knowledge/search	backend/app/knowledge/retrieval/vector.py	KnowledgeEmbeddingModel(pgvector extension)	tests/integration/test_hybrid_retrieval.py	**RUNTIME-VERIFIED**

Requirement ID	Requirement Title & Description	Frontend Route & Component	API Endpoint	Implementing Backend Service	Database Model / Provider	Verification Test File	Status
REQ-KNOW-03	**Relational Knowledge Graph**Entity and relationship storage in PostgreSQL with neighborhood traversal.	/ knowledgeapps/web/src/app/knowledge/page.tsx	GET /api/v1/knowledge/entities/{id}/relationships	backend/app/knowledge/graph/repository.py	KnowledgeGraphNodeModel KnowledgeGraphEdgeModel	tests/integration/test_hybrid_retrieval.py	**RUNTIME-VERIFIED**
REQ-KNOW-04	**Hybrid Retrieval Fusion (RRF)**Fuses lexical BM25, dense vector, and graph rankings via reciprocal rank fusion.	/ knowledgeapps/web/src/app/knowledge/page.tsx	POST /api/v1/knowledge/search	backend/app/knowledge/retrieval/hybrid.py	HeuristicReranker	tests/integration/test_hybrid_retrieval.py	**TEST-VERIFIED**
REQ-AGENT-01	**Multi-Agent Orchestrator** Coordinates 5 specialized agents with bounded 5-step execution planning.	/agentsapps/web/src/app/agents/page.tsx	POST /api/v1/agents/chat	backend/app/agents/orchestrator.py	ToolRegistry	tests/api/test_agents_api.py	**TEST-VERIFIED**
REQ-SEC-01	**SSRF Ingestion Protection**Blocks private RFC1918 IPs and cloud metadata URLs during source fetch.	/admin/sourcesapps/web/src/app/admin/sources/page.tsx	POST /api/v1/knowledge/ingestion/jobs	backend/app/knowledge/ingestion/fetcher.py	DNS inspection socket	tests/security/test_ssrf_defense.py	**TEST-VERIFIED**
REQ-SEC-02	**Token Bucket Rate Limiter**Enforces 200 req capacity with 5.0 tokens/sec refill on all incoming traffic.	All endpoints	All endpoints	backend/app/common/rate_limiter.py	In-memory token bucket	tests/security/test_rate_limiter.py	**TEST-VERIFIED**
REQ-AUD-01	**Tamper-Evident Audit Trail**Captures structured JSON operational and security logs with timestamps.	/admin/auditapps/web/src/app/admin/audit/page.tsx	GET /api/v1/audit/events	backend/app/common/audit.py	AuditEventModel	tests/unit/test_audit.py	**RUNTIME-VERIFIED**

2. Requirement Status Summary Statistics

- **Total Tracked Core Requirements:** 22
- **Runtime-Verified in Live Production:** 8 (36.4%)
- **Test-Verified via Automated CI Test Suite:** 12 (54.5%)
- **Source-Verified (Implementation Confirmed):** 2 (9.1%)
- **Planned / Absent:** 0 (0.0%)
- **Overall Specification Implementation Completeness: 100% (22 / 22 Core Functional Requirements Implemented)**

Chapter 15 — Troubleshooting, Known Limitations and Technical Debt

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Technical Debt & Production Blockers Inventory

In accordance with strict forensic standards, this chapter documents the real, unvarnished architectural trade-offs, constraints, and known limitations currently present in CivicSphere AI.

Issue ID	Category	Severity	Summary	Current Status	Remediation Required
LIMIT-01	AI Engine	**HIGH (Prod Blocker)**	Production uses deterministic token-hash mock embeddings (mock)	Active in Prod	Provision commercial OpenAI / Cohere API key
LIMIT-02	Cloud DB	**MEDIUM**	Supabase free-tier projects automatically pause after 7 days of inactivity	Restored / Active	Upgrade to Supabase Pro or dedicated AWS RDS instance
LIMIT-03	Serverless	**LOW**	Vercel Python serverless cold starts experience ~1.5s - 2.5s initial hydration latency	Active in Prod	Deploy on dedicated container (Railway / AWS ECS / Render)
LIMIT-04	Graph DB	**LOW**	Neo4j enterprise driver is optional; production relies on PostgreSQL relational graph	Retained Fallback	Maintain relational graph or provision hosted AuraDB Neo4j
LIMIT-05	Rate Limit	**LOW**	Rate limiting is currently in-memory per container rather than Redis cluster	Active in Prod	Point REDIS_URL to an Upstash / ElastiCache Redis cluster
LIMIT-06	Repo Cleanup	**INFO**	Legacy root civicsphere_knowledge.db exists from local development	Harmless Artifact	Remove file from local working tree (ignored by git)

2. In-Depth Technical Limitations & Remediation

2.1 LIMIT-01: Commercial Embedding Key Requirement

- **Description:** In `backend/app/common/config.py`, `EMBEDDING_PROVIDER` is set to `"mock"` in production. While the mock provider uses deterministic feature hashing with stopword removal for exact keyword-semantic matching (allowing accurate retrieval for indexed terms like “Article 21”, “Section 7 RTI”, “Consumer Forum”), it lacks the high-dimensional latent semantic generalization of dense neural models like OpenAI `text-embedding-3-small` or Cohere `embed-english-v3.0`.
- **Root Cause:** A commercial OpenAI or Cohere API key has not been funded or attached to the Vercel production environment variables.
- **Evidence:** `backend/app/common/providers/embedding.py::get_embedding_provider()` falls back to `MockEmbeddingProvider(dimension=384)`.
- **Workaround:** Seed sources contain rich lexical metadata, allowing hybrid BM25 + graph traversal to maintain high retrieval precision (≥ 0.88).
- **Resolution:**
  1. Add `OPENAI_API_KEY="sk-..."` in Vercel project environment variables.
  2. Set `EMBEDDING_PROVIDER="openai"` in Vercel production settings.
  3. Trigger automated re-indexing via `POST /api/v1/knowledge/ingestion/jobs`.

2.2 LIMIT-02: Supabase Free-Tier Auto-Pausing Behavior

- **Description:** The production PostgreSQL database is provisioned on the Supabase free tier. Supabase free projects enter a paused state after 7 consecutive days of inactivity. When paused, the transaction pooler rejects connections on port 6543 until manually unpaused via the Supabase dashboard.

- **Evidence:** During earlier release testing, the database entered dormancy and returned `ConnectionRefusedError` until unpaused by the project owner.
- **Workaround:**
  1. A daily automated ping script (`infrastructure/scripts/verify_hosted_postgres.py`) can be scheduled via GitHub Actions cron to prevent dormancy.
  2. For local evaluations and viva demonstrations, the local Docker PostgreSQL stack (`infrastructure/docker/docker-compose.yml`) provides zero-latency offline fallback.
- **Resolution:** For commercial enterprise operation, migrate from Supabase free tier to a dedicated AWS RDS PostgreSQL instance or Supabase Pro tier with persistent compute.

2.3 LIMIT-03: Serverless Python Cold-Start Latency

- **Description:** Vercel serverless functions running Python 3.12 spin down to 0 replicas when traffic is idle. On a fresh invocation:
  1. The serverless container initializes Python 3.12 runtime and downloads unbundled wheels (`pygments`, `sqlalchemy`, `asyncpg`).
  2. `lifespan()` executes, connecting to PostgreSQL and hydrating 34 graph nodes and 12 vector chunks into memory cache.
  – This results in a cold-start response latency of ~1.8 to 2.5 seconds for the very first request. Subsequent warm requests respond in under 80 ms.
- **Mitigation Implemented:** Commit `894f27d` added `connect_args["timeout"] = 3.0` and `asyncio.wait_for(init_models(), timeout=2.5)` to ensure the serverless container never hangs indefinitely waiting for a slow pooler.
- **Resolution:** For zero cold-start latency, host the FastAPI backend on a persistent container runtime (e.g., Render, Railway, AWS ECS, or DigitalOcean App Platform) using the provided `infrastructure/docker/Dockerfile.backend`.

2.4 LIMIT-04: Relational Graph vs. Neo4j Cypher Storage

- **Description:** While `backend/app/knowledge/graph/repository.py` supports native Neo4j Bolt connections (`neo4j://localhost:7687`), serverless edge environments lack a persistent Neo4j cluster unless an external cloud service (AuraDB) is provisioned.
- **Current Behavior:** The system uses a durable relational graph representation in PostgreSQL (`knowledge_graph_nodes` and `knowledge_graph_edges` tables) with recursive CTE traversal.
- **Impact:** Relational graph traversal in PostgreSQL provides identical 1-hop and 2-hop neighborhood exploration for civic acts with zero external dependency overhead.

2.5 LIMIT-05: In-Memory Token Bucket Rate Limiting

- **Description:** The current `RateLimitMiddleware` maintains client token buckets in process memory. Across multiple concurrent Vercel serverless function instances, each container tracks rate limits independently rather than sharing state.
- **Impact:** In high-concurrency environments, a distributed denial of service attack could distribute requests across multiple cold containers, effectively multiplying the allowed request quota.
- **Resolution:** Connect the middleware to the provisioned Redis container (`REDIS_URL`) or an Upstash Serverless Redis instance for global atomic rate limiting.

Chapter 16 — Developer Handover and Maintenance Guide

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026

1. Core Codebase Modules to Understand First

When onboarding to the CivicSphere AI codebase, examine these core files in order:

1. **Backend Entrypoint & Lifespan:** `backend/app/main.py`
   Why: Understands CORS, security headers, database cold-start hydration, and router mounting.
2. **Configuration & Security Settings:** `backend/app/common/config.py`
   Why: Understands Pydantic Settings validation, required secret formats, and provider flags.
3. **Canonical Schemas & Contracts:** `packages/schemas/contracts.py`
   Why: Defines the universal `ApiResponse[T]` envelope, `EvidencePack`, and `LegalAnswer` interfaces.
4. **Knowledge Ingestion Pipeline:** `backend/app/knowledge/ingestion/pipeline.py`
   Why: Traces the 13-stage ingestion sequence from statutory text to dual vector/graph indexing.
5. **Universal Frontend API Client:** `apps/web/src/lib/api-client.ts`
   Why: Shows how the Next.js client propagates httpOnly cookies and `X-CSRF-Token` headers.

2. Step-by-Step Developer Extension Guides

2.1 How to Add a New Frontend Page

1. Create a new directory and `page.tsx` file in `apps/web/src/app/` (e.g. `apps/web/src/app/consumer-disputes/page.tsx`).
2. Mark the file with `"use client";` if utilizing React hooks (`useState`, `useEffect`).
3. Import the shared UI components and API client:

```
"use client";
import React, { useState } from "react";
import { api } from "@/lib/api-client";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";

export default function ConsumerDisputesPage() {
  return (
    <div className="max-w-5xl mx-auto p-6">
      <h1 className="text-2xl font-bold">Consumer Dispute Navigator</h1>
    </div>
  );
}
```

4. Add a navigation link in `apps/web/src/components/Navigation.tsx`.
5. Verify linting passes: `npm run lint --workspace=civicsphere-web`.

2.2 How to Add a New Backend API Endpoint

1. Open the relevant domain router under `backend/app/` (or create a new router file, e.g. `backend/app/consumer/routes.py`).
2. Define the Pydantic request model:

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field
from packages.schemas.contracts import ApiResponse
from backend.app.auth.dependencies import get_current_user
from backend.app.auth.models import UserModel

router = APIRouter(prefix="/consumer", tags=["Consumer Protection"])

class ClaimCalculationRequest(BaseModel):
    amount_inr: float = Field(ge=0, description="Total consideration paid in INR")

@router.post("/calculate-forum", response_model=ApiResponse[dict])
async def calculate_jurisdictional_forum(
    payload: ClaimCalculationRequest,
    current_user: UserModel = Depends(get_current_user),
):
    """Determines District, State, or National commission under Consumer Protection Act 2019."""
    if payload.amount_inr <= 5000000:
        forum = "District Consumer Disputes Redressal Commission (Section 34)"
    elif payload.amount_inr <= 20000000:
        forum = "State Consumer Disputes Redressal Commission (Section 47)"
    else:
```

```
        forum = "National Commission / NCDRC (Section 58)"
    return ApiResponse(data={"forum": forum, "amount": payload.amount_inr}, confidence=1.0)
```

3. Register the router in `backend/app/main.py`:

```
from backend.app.consumer.routes import router as consumer_router
app.include_router(consumer_router, prefix=settings.API_PREFIX)
```

4. Write an automated test in `backend/tests/api/test_consumer_api.py`.

2.3 How to Add a New Database Model & Migration

1. Declare the SQLAlchemy model inheriting from `backend.app.common.database.Base`:

```
from sqlalchemy import Column, String, Float, DateTime
from sqlalchemy.sql import func
from backend.app.common.database import Base

class ConsumerGrievanceModel(Base):
    __tablename__ = "consumer_grievances"

    id = Column(String(36), primary_key=True)
    claim_amount = Column(Float, nullable=False)
    forum_assigned = Column(String(255), nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

2. Generate an Alembic migration revision:

```
alembic revision --autogenerate -m "add_consumer_grievances_table"
```

3. Inspect the generated migration script in `backend/migrations/versions/`.
4. Apply the migration:

```
alembic upgrade head
```

2.4 How to Add a New Statutory Legal Source

1. Open `data/seed/seed_data.py`.
2. Append a dictionary to `SEED_SOURCES` with the verified statutory text:

```
{
    "title": "Arbitration and Conciliation Act, 1996",
    "publisher": "Ministry of Law and Justice",
    "official_url": "https://www.indiacode.nic.in/handle/123456789/2180",
    "jurisdiction": "IN",
    "source_type": "ACT",
    "trust_level": SourceTrustLevel.OFFICIAL_LEGISLATION,
    "publication_date": "1996-08-16",
    "effective_date": "1996-08-22",
    "raw_text": """
    Section 7. Arbitration agreement.
    (1) In this Part, "arbitration agreement" means an agreement by the parties to submit to arbitration all
or certain disputes which have arisen or which may arise between them in respect of a defined legal
relationship, whether contractual or not.
    (2) An arbitration agreement may be in the form of an arbitration clause in a contract or in the form of
a separate agreement.
    (3) An arbitration agreement shall be in writing.
    """
}
```

3. Trigger re-seeding via Python CLI:

```
python -c "import asyncio; from data.seed.seed_data import seed_knowledge_base;
asyncio.run(seed_knowledge_base())"
```

4. Verify the new act appears in `GET /api/v1/legal/acts` and is searchable in `/legal`.

2.5 How to Add a New AI Embedding or LLM Provider

1. Open `backend/app/common/providers/embedding.py`.
2. Implement the `EmbeddingProvider` abstract base class:

```
class CohereEmbeddingProvider(EmbeddingProvider):
    def __init__(self, api_key: str, model: str = "embed-english-v3.0", dimension: int = 384):
        self.api_key = api_key
        self.model = model
        self._dimension = dimension

    def get_dimension(self) -> int:
        return self._dimension

    async def embed_text(self, text: str) -> List[float]:
        # Call Cohere async client and return L2-normalized vector
        ...
```

3. Update the factory function `get_embedding_provider()` to handle `provider_name == "cohere"`.
4. Add the corresponding environment variable `COHERE_API_KEY` to `backend/app/common/config.py`.

3. Important Invariants & Coding Conventions

- **Universal Response Envelope:** All API routes must return `ApiResponse[T]` (from `packages.schemas.contracts`). Never return raw unbound dictionaries directly from endpoints.
- **Never Log Sensitive Credentials:** Never print `JWT_SECRET_KEY`, database passwords, or auth cookies in log streams.
- **Strict Anti-Hallucination Disclaimers:** Any newly added AI guidance endpoint must include the statutory disclaimer clarifying that the output is civic statutory intelligence and not formal legal counsel.
- **Transaction Statement Cache:** When configuring asyncpg with Supabase, always maintain `statement_cache_size: 0` in `connect_args`.

Chapter 17 — Academic Demonstration and Viva Technical Guide

Document Version: 1.0.0
Repository: `Khushbu112005/Finalyear_MajorProject`
Baseline cited by source chapter (not independently reverified here): `b566c93`
Investigation Date: October 28, 2026
Audience: Final-Year Major Project Examination Panel, Project Evaluators & Defense Candidates

1. Project Summary for Evaluation Panel

Project Title: CivicSphere AI — AI-Powered Civic and Legal Intelligence Platform
Domain: Artificial Intelligence, Multi-Agent Systems, Natural Language Processing, Full-Stack Engineering, Legal Informatics
Primary Contribution: An evidence-grounded, anti-hallucinatory civic intelligence architecture tailored to the Indian legal system. The platform combines a 13-stage statutory ingestion pipeline, dual vector-graph indexing, hybrid RAG retrieval with Reciprocal Rank Fusion, five specialized agent personas, and strict security isolation (RBAC, IDOR, SSRF defense, and double-submit CSRF).

2. 10-Step Live Viva Demonstration Walkthrough

During your final-year project viva, follow this exact sequence to demonstrate all 6 core modules and prove genuine database persistence:

```
[Step 1: System Health & Readiness]
  -> Navigate to: https://civicsphere-backend.vercel.app/api/v1/health/ready
  -> Evidence: Point out `"ready": true`, `"graph_nodes": 34`, `"vector_records": 12`, `"persistence":
"postgresql_pgvector"`

[Step 2: Platform Landing Page]
  -> Navigate to: https://civicsphere-web.vercel.app/
  -> Evidence: Highlight the 6 integrated module launch cards and the clean responsive Next.js 15 App Router design.

[Step 3: User Registration & Session Security]
  -> Navigate to: /register
  -> Action: Register a new account (e.g. `viva.evaluator@example.com`, Citizen role).
  -> Evidence: Show automatic issuance of httpOnly JWT cookie and X-CSRF-Token generation.

[Step 4: Statutory Legal Guidance (Module A)]
  -> Navigate to: /legal
  -> Query: "What is the maximum time limit for a Public Information Officer to respond to an RTI request?"
  -> Evidence: Show 10-section structured guidance, 98% confidence score, and exact citation to Section 7(1) of the
RTI Act 2005.

[Step 5: Anti-Hallucination Out-of-Domain Circuit Breaker]
  -> Query: "How do quantum fluctuations affect Hawking radiation in black holes?"
  -> Evidence: System immediately returns 0.0% confidence, triggers the out-of-domain guardrail, and refuses to
hallucinate legal rules.

[Step 6: Government Scheme Navigator (Module B)]
  -> Navigate to: /government
  -> Query: "I need financial support to construct an urban house for my economically weaker family."
  -> Input Demographics: Income < 3 Lakhs, Urban Maharashtra.
  -> Evidence: System recommends Pradhan Mantri Awas Yojana (PMAY-U) with exact CLSS subsidy calculations and
eligibility rules.

[Step 7: Citizen Case Workspace & IDOR Protection]
  -> Navigate to: /cases
  -> Action: Create a new legal case matter titled "Consumer Defect Claim against Electronics Vendor".
  -> Evidence: Case record saved to PostgreSQL with unique UUID, tracking status as `OPEN`. Show that another user
cannot view this case without authorization.

[Step 8: Intelligent Document Processing & OCR (Module D)]
  -> Navigate to: /documents
  -> Action: Upload a sample legal notice PDF (`uploads/doc_0a34cc1fee634178a8ebbff8cef7694e.pdf`).
  -> Evidence: Show the 7-stage processing pipeline executing magic-byte validation, PyMuPDF text extraction, and
legal document classification.

[Step 9: Interactive Knowledge Graph Explorer (Module C)]
  -> Navigate to: /knowledge
  -> Action: Explore the neighborhood of the entity "Central Information Commission".
  -> Evidence: Show connected statutory nodes: `Right to Information Act 2005` (ESTABLISHES), `Second Appellate
Authority` (POWERS).

[Step 10: Administrative Compliance & Audit Trail (Phase 10)]
  -> Navigate to: /admin/audit (or query `GET /api/v1/audit/events`)
  -> Evidence: Display structured JSON event logs capturing the user registration, ingestion job, and query execution
timestamps.
```

3. High-Probability Viva Defense Questions & Model Answers

Q1: What makes CivicSphere AI different from asking ChatGPT or Claude a legal question?

Model Answer:
"Generic LLMs suffer from severe legal hallucinations: they invent non-existent statutory sections, cite overruled judgments, and apply foreign (US/UK) laws to Indian contexts. CivicSphere AI implements an **evidence-first, anti-hallucination architecture**:

1. It queries a closed, verified knowledge repository of official Indian enactments (e.g., RTI Act, DPDP Act, Consumer Protection Act) published via India Code.
2. An out-of-domain circuit breaker automatically rejects non-legal or out-of-scope queries with confidence `0.0`.
3. Every response is synthesized exclusively from retrieved statutory chunks and must include verified section-level citations with official legislative URLs."

Q2: Why did you choose a Hybrid Retrieval architecture (Lexical + Vector + Graph) instead of standard vector-only RAG?

Model Answer:
"Vector-only RAG excels at fuzzy semantic matching, but it struggles with exact statutory references. For example, a citizen asking about *'Section 7(1) proviso'* requires exact lexical matching (BM25) because dense embeddings often blur fine subsection distinctions. Conversely, legal reasoning is inherently relational: an Act *establishes* a Commission, which *adjudicates* Appeals against an Officer. By fusing **BM25 lexical search**, **pgvector dense cosine similarity**, and **graph neighborhood traversal** using **Reciprocal Rank Fusion (RRF with $k = 60$)**, we achieve both high semantic recall and pinpoint statutory precision."

Q3: How do you guarantee durable persistence in serverless environments like Vercel?

Model Answer:
"Vercel serverless functions are ephemeral—in-memory objects and local `/tmp` files are destroyed between invocations. To ensure permanent persistence:

1. All application state (users, cases, documents, knowledge chunks, graph nodes, and pgvector embeddings) is stored in a hosted PostgreSQL 17.6 database on Supabase via Alembic migrations.
2. In `backend/app/main.py::lifespan()`, we implement **Cold-Start Hydration**: upon container startup, the backend connects to PostgreSQL via `asyncpg`, loads the registered source metadata, and hydrates the in-memory graph repository in under 800 milliseconds.
3. We set `statement_cache_size = 0` and connection timeouts to ensure compatibility with Supabase's transaction pooler."

Q4: Explain the security threat model of the platform.

Model Answer:
"CivicSphere AI is engineered according to OWASP Top 10 and multi-tenant security principles:

- **Authentication:** Passwords hashed with `bcrypt` (12 rounds); sessions managed via `httpOnly`, `Secure` JWT cookies with 24-hour expiration.
- **CSRF Defense:** Double-submit cookie pattern with `X-CSRF-Token` header verification on all mutations.
- **IDOR Protection:** Case workspace records strictly enforce `citizen_id == current_user.id` or assigned counsel role.
- **SSRF Defense:** During remote source ingestion, target URLs are validated to block private RFC 1918 IPs and cloud metadata addresses (`169.254.169.254`).
- **File Upload Protection:** Uploaded PDFs are validated using `%PDF-` magic-byte checks and scanned for active JavaScript/executable streams before parsing."

Q5: How do the specialized AI agents coordinate their work?

Model Answer:
"The platform features a Multi-Agent Orchestrator in `backend/app/agents/orchestrator.py` managing 5 specialists: Legal, Knowledge, Government, Document, and Citation agents. The orchestrator:

1. Formulates a bounded execution plan (maximum 5 steps to prevent runaway loops).
2. Dispatches sub-tasks to agents, each executing sandboxed tools from a registered `ToolRegistry` with strict Pydantic parameter validation.
3. Synthesizes an unified response, attaching full execution logs and citation verification proofs."

Q6: What are the current limitations of the platform?

Model Answer:
“To maintain academic integrity, we openly document two primary constraints:

1. **Mock Embeddings in Production:** Because commercial OpenAI/Cohere API keys require ongoing billing, production currently runs with a deterministic token-hash mock embedding provider (384 dimensions). This provides exact keyword-semantic matching for indexed acts, but high-dimensional dense neural embeddings will be enabled once commercial credentials are provisioned.
2. **Database Inactivity Pausing:** On the free Supabase tier, the database automatically pauses after 7 days of idle time. For our viva defense, we maintain a containerized local Docker PostgreSQL stack on port 5432 to provide an instant, zero-latency backup.”

4. Evaluation Checklist for Viva Examiners

Criterion	Evaluation Standard	Evidence in Project
Problem Formulation	Clear, societal impact addressing Indian civic/legal accessibility	10-section legal guidance, RTI/DPDP/Consumer acts
System Architecture	Robust, modular, and decoupled design	Modular Monolith FastAPI backend + Next.js 15 client
Database Design	Normalized schema, ACID compliance, vector storage	PostgreSQL 17 + pgvector, Alembic migrations 0001-0003
AI / NLP Rigor	Advanced RAG with hybrid search and anti-hallucination	BM25 + Vector + Graph fusion via RRF; circuit breaker
Code Quality & QA	Comprehensive test coverage with zero failures	91 passing tests in Pytest; ESLint 0 errors; TypeScript typed
Security Engineering	Proactive mitigation of common web vulnerabilities	RBAC, httpOnly JWT, CSRF token, SSRF defense, IDOR checks
Live Demonstration	Functioning deployment accessible over the public internet	civicsphere-web.vercel.app & civicsphere-backend.vercel.app

Appendix A — Completion Report Claims and Reconciliation

The following is a structured digest of the supplied Antigravity completion output. It records what that output claims; it is not an independent audit result.

A.1 Claimed dossier generation and Git operation

The completion report states that `docs/project-dossier/` was created with a README and chapters 00–17, and that the dossier was committed and pushed to `origin/main`. It identifies `e28d77e` as the latest baseline commit. The 18 chapter files supplied separately instead cite `b566c93`. Because both values appear in the source packet and no authenticated Git remote check was executed as part of this compilation, the currently authoritative remote revision remains **not independently verified here**.

A.2 Reported discovery counts

Item	Completion report claim	Interpretation
Frontend routes	28	Reported repository inventory; re-count from actual apps/web/src/app/ before finalizing the exact current count.
Backend endpoints	41	Consistent with the API table's numbered entries 1–41, although the chapter introduction originally said 32.
Database tables	10	Reported inventory: users, cases, documents, government services, audit events, knowledge sources, chunks, embeddings, graph nodes and graph edges. Verify against current migrations.
Environment variable identifiers	29	Reported audit total. Chapter 08 remains the detailed inventory; current deployed configuration status is not independently verified by this compilation.
Embedding implementations	4	Mock, OpenAI, Gemini, Hugging Face (as described). Implementation does not imply active runtime use.
LLM implementations	4	Mock, Anthropic, OpenAI, Google/Gemini (as described). Implementation does not imply active runtime use.
Rerankers	3	Heuristic, Cohere and cross-encoder variants are described.
Tests	91 passed, 1 skipped, 0 failed	Reported execution result. The chapters and output report run times vary.
Evaluation data files	4 in the completion summary	Chapter 07 lists five named files when the aggregate dataset and held-out file are both counted.
Synthetic PDF fixtures	36	Completion report claim; verify by counting current files in the repository.

A.3 Deployment and persistence statements

The completion report names these deployed URLs:

- Frontend: `https://civicsphere-web.vercel.app`
- Backend: `https://civicsphere-backend.vercel.app`

It reports HTTP 200 for frontend/backend health endpoints and describes Supabase PostgreSQL 17.6 plus pgvector with 34 hydrated graph nodes and 12 vectors. Elsewhere, the technical limitations chapter warns that the free-tier database can pause, and that database writes may fail during that period. These statements may correspond to different times. Before a live demo, verify actual availability, registration, a non-destructive write/read round trip, and persistence across a fresh process.

A.4 AI evaluation and provider disclosure

The completion report reproduces high evaluation scores, including MRR 1.0000, Precision@1 1.0000, Precision@5 0.9500, Recall@5 0.9200, NDCG@5 0.9600, groundedness 98%, citation correctness 100%, and an in-process average latency of 0.81 ms. These figures should be cited only as reported benchmark results, together with the benchmark command, dataset, code revision and raw output. The source packet also explicitly states that production uses deterministic mock embeddings and mock LLM behaviour unless provider configuration is changed. Therefore, the scores must not be interpreted as proof that the deployed system is running a live neural semantic retrieval pipeline.

A.5 Consolidated readiness assessment

The original completion output recommends academic defense and live demonstration as GO, while placing enterprise/commercial production on HOLD because commercial provider keys and stable database operation remain concerns. This dossier preserves that recommendation as a source-reported conclusion; it does not independently certify the live application.

Readiness area	Consolidated assessment	Basis and condition
Academic submission package	**Documentation consolidated; institution-specific compliance not assessed**	The supplied README, 18 technical chapters and completion report have been brought together here. A department-specific report template, originality requirements, signatures, and submission portal requirements were not included in the supplied packet and therefore are not certified by this dossier.
Live demonstration	**CONDITIONAL — run the pre-demo checklist**	The source reports describe successful health and legal-query checks, but the execution trace also records cancelled or failed live probes and database connectivity failures. Test registration/login, a supported legal query with source citations, the unsupported-query safeguard, and a safe persistence round-trip on the actual demonstration network before calling it operational.
Production AI quality	**NOT VERIFIED as real semantic AI**	The supplied technical chapters explicitly disclose that the reported production configuration uses mock embeddings and mock LLM behaviour unless external providers are configured. Results from fixtures, token-hash embeddings or an in-process benchmark must not be described as proof of a live neural retrieval pipeline.
Durable database operations	**CONDITIONAL — availability must be checked at use time**	Supabase availability/write behaviour is described inconsistently across supplied materials and may change with project state. Confirm a safe synthetic write/read round-trip and persistence in the selected environment. A successful health endpoint alone is insufficient.
Enterprise/commercial production	**HOLD / NO-GO on the supplied evidence**	Real provider configuration, stable database operation, live end-to-end verification and confirmation that any genuinely exposed credentials were rotated must be demonstrated before this higher standard can be claimed.

This consolidated document confirms that the technical documentation materials are organized in a single reference, while preserving source claims and openly identifying disagreements. It does not certify that all application features were independently executed in this compilation. Use Appendix C to capture fresh evidence on the exact revision and environment intended for evaluation.

Appendix B — Source Packet Inventory

This consolidated dossier was constructed from the following supplied files:

1. `00_MASTER_PROJECT_DOSSIER.md`
2. `01_PROJECT_ORIGIN_REQUIREMENTS_AND_DEVELOPMENT_HISTORY.md`
3. `02_COMPLETE_REPOSITORY_AND_TECH_STACK_INVENTORY.md`
4. `03_FEATURES_PAGES_COMPONENTS_AND_USER_WORKFLOWS.md`
5. `04_COMPLETE_BACKEND_API_CATALOGUE.md`
6. `05_DATABASE_SCHEMA_MIGRATIONS_AND_PERSISTENCE.md`
7. `06_AI_ML_RAG_AGENTS_AND_PROMPT_ARCHITECTURE.md`
8. `07_DATASETS_KNOWLEDGE_BASE_AND_EXTERNAL_DATA_SOURCES.md`
9. `08_API_KEYS_ENVIRONMENT_VARIABLES_AND_EXTERNAL_INTEGRATIONS.md`
10. `09_AUTHENTICATION_AUTHORIZATION_AND_SECURITY_ARCHITECTURE.md`
11. `10_SYSTEM_ARCHITECTURE_AND_END_TO_END_DATA_FLOWS.md`
12. `11_INSTALLATION_CONFIGURATION_AND_BUILD_FROM_SCRATCH.md`
13. `12_TESTING_QA_BENCHMARKS_AND_VERIFICATION_EVIDENCE.md`
14. `13_DEPLOYMENT_CICD_INFRASTRUCTURE_AND_OPERATIONS.md`
15. `14_FEATURE_TRACEABILITY_AND_IMPLEMENTATION_STATUS.md`
16. `15_TROUBLESHOOTING_KNOWN_LIMITATIONS_AND_TECHNICAL_DEBT.md`
17. `16_DEVELOPER_MAINTENANCE_AND_FUTURE_EXTENSION_GUIDE.md`
18. `17_ACADEMIC_DEMONSTRATION_AND_VIVA_TECHNICAL_GUIDE.md`
19. `README(20261028-063005).md`
20. `Pasted text(20261028-063535).txt` — Antigravity dossier-completion report and execution trace.

The README's navigation and stated readiness banner have been incorporated into the master introduction and reconciliation register rather than duplicated verbatim. The completion output is summarized in Appendix A to preserve its claims while clearly separating those claims from independently reproduced evidence.

Appendix C — Pre-Demonstration Verification Checklist

Run this checklist against the actual final revision and the environment you intend to use for the viva. Use a clean test account and synthetic data; never expose credentials on screen.

C.1 Revision and environment

- ☐ Confirm the exact Git commit and clean/expected working-tree status.
- ☐ Confirm which frontend version is installed from the lockfile and package manifest.
- ☐ Confirm the configured frontend API base URL points to the intended backend.
- ☐ Confirm required environment variables are set without printing values.
- ☐ Confirm any exposed credentials have been rotated and deployments use the rotated credentials.

C.2 Application access and core features

- ☐ Open the frontend and navigate to the required routes.
- ☐ Register a synthetic citizen/researcher account if registration is part of the demo.
- ☐ Log in, inspect the protected user route, and log out.
- ☐ Submit a supported Indian civic/legal question and inspect its actual source citations.
- ☐ Submit a clearly unsupported/out-of-domain question and verify the insufficient-evidence behaviour.
- ☐ Open the government-service page and test a representative scheme query with clearly synthetic inputs.
- ☐ Upload a benign, synthetic PDF and verify the actual extraction response, if this feature is being demonstrated.
- ☐ Create/read/update a synthetic case only if the database is healthy and the demonstration uses an approved test environment.
- ☐ Inspect knowledge graph/source views that are part of the planned demo.

C.3 Persistence and AI honesty

- ☐ Check backend health and readiness, but do not use health alone as proof of database writes.
- ☐ Perform one safe write/read round trip using synthetic data.
- ☐ Verify data persists across a fresh application process, where feasible.
- ☐ Confirm which embedding and LLM providers are genuinely active.
- ☐ Label mock-mode behaviour honestly if no real provider is configured.
- ☐ Do not present fixture-derived scores as live production performance.

C.4 Final checks

- ☐ Run the selected final regression suite and retain the raw output.
- ☐ Verify the current deployment URLs from a browser on the presentation machine/network.
- ☐ Remove test records created specifically for the demo by exact IDs only, or use a disposable database.
- ☐ Prepare a fallback plan using the documented local Docker environment if the hosted database is unavailable.
- ☐ Keep the known-limitations chapter accessible and answer evaluator questions accurately.

Decision rule: if registration, the primary legal-query workflow, citation rendering, or required data persistence fails, do not claim a fully operational live demo. Switch to a tested local environment or disclose the limitation before the demonstration.